

Backend Kommentar-Dokumentation

Generiert aus C# XML Documentation Comments (///)

Program

Configures and starts the ASP.NET Core application host, including dependency registration, middleware pipeline, and endpoint mapping.

Task_Weaver_Backend.Controllers.BlockerController

Controller for managing user calendar blockers.

M:Task_Weaver_Backend.Controllers.BlockerController.#ctor(Task_Weaver_Backend.Services.IBlockerService)

Initializes a new instance of the

Parameter

- blockerService : The blocker service.

Task_Weaver_Backend.Controllers.FirebaseControllerBase

Represents the firebase controller base.

M:Task_Weaver_Backend.Controllers.FirebaseControllerBase.GetFirebaseUid

Extracts the Firebase UID from the authenticated user's claims.

Returns: The Firebase UID.

Exceptions

- T:Task_Weaver_Backend.Models.AppException : Thrown when the token is invalid.

Task_Weaver_Backend.Controllers.OrganizationController

Controller for organization management and administration operations.

M:Task_Weaver_Backend.Controllers.OrganizationController.#ctor(Task_Weaver_Backend.Services.IOrganizationService)

Initializes a new instance of the

Parameter

- organizationService : The organization service.

M:Task_Weaver_Backend.Controllers.OrganizationController.CreateInvite(System.String,Task_Weaver_Backend.Models.DTOS.CreateInviteDto)

Creates an invitation to join the organization. Requires admin role.

Parameter

- orgId : Organization identifier.
- request : Invitation request payload.

Returns: HTTP 201 with invitation token.

M:Task_Weaver_Backend.Controllers.OrganizationController.DeleteInvite(System.String,System.String)

Deletes a pending invitation. Requires admin role.

Parameter

- orgId : Organization identifier.
- inviteId : Invitation identifier.

Returns: HTTP 204 No Content.

M:Task_Weaver_Backend.Controllers.OrganizationController.GetInvites(System.String)

Retrieves all pending invitations for an organization. Requires admin role.

Parameter

- orgId : Organization identifier.

Returns: HTTP 200 with list of pending invitations.

M:Task_Weaver_Backend.Controllers.OrganizationController.GetMembers(System.String)

Retrieves all members of an organization. Requires admin role.

Parameter

- orgId : Organization identifier.

Returns: HTTP 200 with list of members.

M:Task_Weaver_Backend.Controllers.OrganizationController.GetOrganization(System.String)

Retrieves organization details. Requires membership.

Parameter

- `orgId` : Organization identifier.

Returns: HTTP 200 with organization details.

`M:Task_Weaver_Backend.Controllers.OrganizationController.GetOrganizationSettings(System.String)`

Retrieves organization settings. Requires admin role.

Parameter

- `orgId` : Organization identifier.

Returns: HTTP 200 with organization settings.

`M:Task_Weaver_Backend.Controllers.OrganizationController.IsEmailInWhitelist(System.String)`

Performs is email in whitelist.

Parameter

- `email` : The email address.

Returns: The requested is email in whitelist.

`M:Task_Weaver_Backend.Controllers.OrganizationController.RemoveMember(System.String,System.String)`

Removes a member from the organization. Requires admin role.

Parameter

- `orgId` : Organization identifier.
- `userId` : User identifier to remove.

Returns: HTTP 204 No Content.

`M:Task_Weaver_Backend.Controllers.OrganizationController.UpdateMemberRole(System.String,System.String,Task_Weaver_Backend.Controllers.UpdateRoleRequest)`

Updates the role of an existing member. Requires admin role.

Parameter

- `orgId` : Organization identifier.
- `userId` : User identifier to update.
- `request` : Role update request payload.

Returns: HTTP 200 with updated member information.

`M:Task_Weaver_Backend.Controllers.OrganizationController.UpdateOrganizationSettings(System.String,Task_Weaver_Backend.Models.OrganizationSettingsDto)`

Updates organization settings. Requires admin role.

Parameter

- `orgId` : Organization identifier.
- `settings` : Updated settings payload.

Returns: HTTP 200 with updated settings.

Task_Weaver_Backend.Controllers.TaskController

Provides task-related endpoints for the authenticated user.

`M:Task_Weaver_Backend.Controllers.TaskController.#ctor(Task_Weaver_Backend.Services.ITaskService)`

Initializes a new instance of the

Parameter

- `taskService` : Service used for task management operations.

`M:Task_Weaver_Backend.Controllers.TaskController.CreateCalendarTask(Task_Weaver_Backend.Models.DTOS.CreateTaskDto)`

Creates a new calendar task for the authenticated user.

`M:Task_Weaver_Backend.Controllers.TaskController.DeleteCalendarTask(System.String)`

Deletes a calendar task for the authenticated user.

`M:Task_Weaver_Backend.Controllers.TaskController.GetCalendarTasks`

Returns all calendar tasks for the authenticated user.

`M:Task_Weaver_Backend.Controllers.TaskController.MoveTaskChunk(System.String,Task_Weaver_Backend.Models.DTOS.MoveTaskChunkDto)`

Moves a scheduled chunk of a task to a new date/time range.

`M:Task_Weaver_Backend.Controllers.TaskController.PatchCalendarTask(System.String,Task_Weaver_Backend.Models.DTOS.TaskDto)`

Updates an existing calendar task for the authenticated user. Only submitted fields are applied. Supports the optional

`M:Task_Weaver_Backend.Controllers.TaskController.PatchCalendarTaskDates(System.String,System.Collections.Generic.List{Task_Weaver_Backend.Models.DTOS.TaskDto})`

Replaces the Dates array of a calendar task.

Task_Weaver_Backend.Controllers.UpdateRoleRequest

Request DTO for updating a member's role.

`P:Task_Weaver_Backend.Controllers.UpdateRoleRequest.IsAdmin`
Gets or sets whether the member should have admin privileges.

Task_Weaver_Backend.Controllers.UserController

Provides user profile and organization membership endpoints for the authenticated user.

`M:Task_Weaver_Backend.Controllers.UserController.#ctor(Task_Weaver_Backend.Services.IUserService)`
Performs user controller.

Parameter

- `userService` : The user service.

`M:Task_Weaver_Backend.Controllers.UserController.GetMyOrganizations`
Returns all organizations assigned to the authenticated user.

`M:Task_Weaver_Backend.Controllers.UserController.LeaveOrganization(System.String)`
Allows the authenticated user to voluntarily leave an organization.

Parameter

- `orgId` : The organization identifier.

Task_Weaver_Backend.Controllers.WorkProfileController

Controller for managing user work profiles, including working schedules and chunk settings.

`M:Task_Weaver_Backend.Controllers.WorkProfileController.#ctor(Task_Weaver_Backend.Services.IWorkProfileService)`
Initializes a new instance of the

Parameter

- `workProfileService` : The work profile service.

`M:Task_Weaver_Backend.Controllers.WorkProfileController.CreateSchedule(Task_Weaver_Backend.Models.WorkingScheduleDto)`
Creates a new working schedule for the authenticated user.

Parameter

- `schedule` : The schedule data to create.

Returns: The created schedule with its generated ID.

Exceptions

- `T:Task_Weaver_Backend.Models.AppException` : Thrown when validation fails or overlap is detected.

`M:Task_Weaver_Backend.Controllers.WorkProfileController.DeleteSchedule(System.String)`
Deletes a working schedule for the authenticated user.

Parameter

- `scheduleId` : The schedule document ID.

Returns: No content on success.

`M:Task_Weaver_Backend.Controllers.WorkProfileController.GetSchedules`
Retrieves all working schedules for the authenticated user.

Returns: A list of working schedules.

`M:Task_Weaver_Backend.Controllers.WorkProfileController.GetSettings`
Retrieves the global chunk settings for the authenticated user.

Returns: The chunk settings, or null if not configured.

`M:Task_Weaver_Backend.Controllers.WorkProfileController.PatchSchedule(System.String,Task_Weaver_Backend.Models.WorkingScheduleDto)`
Updates an existing working schedule for the authenticated user. Performs a full replacement of the schedule data.

Parameter

- `scheduleId` : The schedule document ID.
- `schedule` : The updated schedule data.

Returns: The updated schedule.

Exceptions

- `T:Task_Weaver_Backend.Models.AppException` : Thrown when validation fails, schedule not found, or overlap is detected.

`M:Task_Weaver_Backend.Controllers.WorkProfileController.PatchSettings(Task_Weaver_Backend.Models.GlobalChunkSettingsDto)`
Updates the global chunk settings for the authenticated user.

Parameter

- `settings` : The new chunk settings.

Returns: The updated chunk settings.

Exceptions

- `T:Task_Weaver_Backend.Models.AppException` : Thrown when settings validation fails.

Task_Weaver_Backend.Extension.ServiceCollectionExtensions

Represents the service collection extensions.

`M:Task_Weaver_Backend.Extension.ServiceCollectionExtensions.AddApplicationCors(Microsoft.Extensions.DependencyInjection.IServiceCollection,Microsoft.Extensions.Configuration.IConfiguration)`
Performs add application cors.

Parameter

- `services` : The services.
- `configuration` : The configuration.

`M:Task_Weaver_Backend.Extension.ServiceCollectionExtensions.AddApplicationServices(Microsoft.Extensions.DependencyInjection.IServiceCollection,Microsoft.Extensions.Configuration.IConfiguration)`
Performs add application services.

Parameter

- `services` : The services.
- `configuration` : The configuration.

`M:Task_Weaver_Backend.Extension.ServiceCollectionExtensions.AddFirebaseAuthentication(Microsoft.Extensions.DependencyInjection.IServiceCollection,Microsoft.Extensions.Configuration.IConfiguration)`
Performs add firebase authentication.

Parameter

- `services` : The services.
- `configuration` : The configuration.

`M:Task_Weaver_Backend.Extension.ServiceCollectionExtensions.AddSwaggerDocumentation(Microsoft.Extensions.DependencyInjection.IServiceCollection,Microsoft.Extensions.Configuration.IConfiguration)`
Performs add swagger documentation.

Parameter

- `services` : The services.

Task_Weaver_Backend.Middleware.ExceptionHandlingMiddleware

Middleware to handle exceptions globally and format them as structured ErrorResponseDto.

`M:Task_Weaver_Backend.Middleware.ExceptionHandlingMiddleware.#ctor(Microsoft.AspNetCore.Http.RequestDelegate,Microsoft.Extensions.Logging.ILogger{Task_Weaver_Backend.Middleware.ExceptionHandlingMiddleware})`
Performs exception handling middleware.

Parameter

- `next` : The next.
- `logger` : The logger.

`M:Task_Weaver_Backend.Middleware.ExceptionHandlingMiddleware.InvokeAsync(Microsoft.AspNetCore.Http.HttpContext)`
Performs invoke async.

Parameter

- `context` : The context.

Returns: A task that represents the asynchronous operation.

Task_Weaver_Backend.Models.AppException

Custom exception for application-specific errors that can be handled by the middleware.

`M:Task_Weaver_Backend.Models.AppException.#ctor(System.String,System.String,System.Int32,System.Object)`
Initializes a new instance of the

Parameter

- `errorCode` : The error code.
- `message` : The message.
- `statusCode` : The status code.
- `details` : The details.

`P:Task_Weaver_Backend.Models.AppException.Details`
Optional field-specific error details or a general reason string.

`P:Task_Weaver_Backend.Models.AppException.ErrorCode`
The specific error code for frontend mapping.

`P:Task_Weaver_Backend.Models.AppException.StatusCode`
The HTTP status code to return.

Task_Weaver_Backend.Models.BlockerDto

Data Transfer Object representing a Blocker in the system.

P:Task_Weaver_Backend.Models.BlockerDto.Dates
List of dates (YYYY-MM-DD) calculated based on recurrence.

P:Task_Weaver_Backend.Models.BlockerDto.Description
A detailed description of the blocker.

P:Task_Weaver_Backend.Models.BlockerDto.End
ISO 8601 end date/time.

P:Task_Weaver_Backend.Models.BlockerDto.Id
The unique identifier of the blocker.

P:Task_Weaver_Backend.Models.BlockerDto.RecurrenceCount
Number of repetitions.

P:Task_Weaver_Backend.Models.BlockerDto.Start
ISO 8601 start date/time.

P:Task_Weaver_Backend.Models.BlockerDto.Title
The title of the blocker.

P:Task_Weaver_Backend.Models.BlockerDto.Type
The type of the blocker (unique, daily, weekly).

Task_Weaver_Backend.Models.BlockerRepository

Firestore-backed repository for calendar blocker operations.

M:Task_Weaver_Backend.Models.BlockerRepository.#ctor(Google.Cloud.Firestore.FirestoreDb)
Initializes a new instance of the
Parameter

- db** : The db.

M:Task_Weaver_Backend.Models.BlockerRepository.CreateCalendarBlockerAsync(System.String,Task_Weaver_Backend.Models.BlockerDto)
Executes create calendar blocker async.
Parameter

- uid** : The user identifier.
- blocker** : The blocker.

Returns: A task that represents the asynchronous operation.

M:Task_Weaver_Backend.Models.BlockerRepository.DeleteCalendarBlockerAsync(System.String,System.String)
Executes delete calendar blocker async.
Parameter

- uid** : The user identifier.
- blockerId** : The blocker identifier.

Returns: A task that represents the asynchronous operation.

M:Task_Weaver_Backend.Models.BlockerRepository.GetCalendarBlockersAsync(System.String)
Executes get calendar blockers async.
Parameter

- uid** : The user identifier.

Returns: A task that resolves to the requested result.

M:Task_Weaver_Backend.Models.BlockerRepository.PatchCalendarBlockerAsync(System.String,System.String,Task_Weaver_Backend.Models.BlockerDto)
Executes patch calendar blocker async.
Parameter

- uid** : The user identifier.
- blockerId** : The blocker identifier.
- blocker** : The blocker.

Returns: A task that represents the asynchronous operation.

Task_Weaver_Backend.Models.DTOs.CreateInviteDto

Represents the create invite dto.

P:Task_Weaver_Backend.Models.DT0s.CreateInviteDto.Email

Gets or sets the email address of the invited user.

Task_Weaver_Backend.Models.DT0s.CreateTaskDto

Represents the create task dto.

P:Task_Weaver_Backend.Models.DT0s.CreateTaskDto.CustomChunkSettings

Gets or sets flow-protection chunk settings.

P:Task_Weaver_Backend.Models.DT0s.CreateTaskDto.Date

Gets or sets the calendar day for the Task (null when type dynamic).

P:Task_Weaver_Backend.Models.DT0s.CreateTaskDto.Deadline

Gets or sets the task deadline. (Null when Task type is not dynamic)

P:Task_Weaver_Backend.Models.DT0s.CreateTaskDto.Description

Gets or sets the task description.

P:Task_Weaver_Backend.Models.DT0s.CreateTaskDto.End

ISO 8601 end date/time. (null when type dynamic)

P:Task_Weaver_Backend.Models.DT0s.CreateTaskDto.EnergyLevel

Gets or sets the cognitive energy level of the task.

P:Task_Weaver_Backend.Models.DT0s.CreateTaskDto.NeedToBeFinishedTaskIds

Gets or sets the identifiers of predecessor tasks. (Null when Task type is not dynamic)

P:Task_Weaver_Backend.Models.DT0s.CreateTaskDto.Organization

Gets or sets the organization/company assignment.

P:Task_Weaver_Backend.Models.DT0s.CreateTaskDto.Priority

Gets or sets the task priority.

P:Task_Weaver_Backend.Models.DT0s.CreateTaskDto.RecurrenceCount

Gets or sets the recurrence count for repeating tasks. (null when type dynamic)

P:Task_Weaver_Backend.Models.DT0s.CreateTaskDto.Start

ISO 8601 start date/time. (null when type dynamic)

P:Task_Weaver_Backend.Models.DT0s.CreateTaskDto.TimeEstimateMinutes

Gets or sets the estimated effort in minutes.

P:Task_Weaver_Backend.Models.DT0s.CreateTaskDto.Title

Gets or sets the task title.

P:Task_Weaver_Backend.Models.DT0s.CreateTaskDto.Type

Gets or sets the task type.

Task_Weaver_Backend.Models.DT0s.JsonDateConverter

JSON converter that serializes DateTime as YYYY-MM-DD date string.

M:Task_Weaver_Backend.Models.DT0s.JsonDateConverter.Read(System.Text.Json.Utf8JsonReader@,System.Type,System.Text.Json.JsonSerializerOptions)

Performs read.

Parameter

- reader : The reader.
- typeToConvert : The type to convert.
- options : The options.

M:Task_Weaver_Backend.Models.DT0s.JsonDateConverter.Write(System.Text.Json.Utf8JsonWriter,System.DateTime,System.Text.Json.JsonSerializerOptions)

Performs write.

Parameter

- `writer` : The writer.
- `value` : The value.
- `options` : The options.

Task_Weaver_Backend.Models.DTOs.MoveTaskChunkDto

Represents the move task chunk dto.

Task_Weaver_Backend.Models.DTOs.OrganizationInformationDto

Data Transfer Object representing an OrganizationData available to members of an Organization in the system.

Task_Weaver_Backend.Models.DTOs.RegisterUserDto

Represents the register user dto.

Task_Weaver_Backend.Models.DTOs.ScheduleDTO

Result object for schedule generation with lossless error reporting. Contains both successfully planned tasks and unresolvable tasks with reasons.

`P:Task_Weaver_Backend.Models.DTOs.ScheduleDTO.GeneratedAt`
Gets or sets the timestamp when this schedule was generated.

`P:Task_Weaver_Backend.Models.DTOs.ScheduleDTO.PlannedTasks`
Gets or sets the list of successfully planned tasks with updated dates.

`P:Task_Weaver_Backend.Models.DTOs.ScheduleDTO.UnresolvableTasks`
Gets or sets the list of tasks that could not be scheduled with reasons.

Task_Weaver_Backend.Models.DTOs.TaskChunkSettingsDto

Describes chunk-size settings that protect flow by preventing excessive task switching.

`P:Task_Weaver_Backend.Models.DTOs.TaskChunkSettingsDto.MaxMinutes`
Gets or sets the maximum chunk size in minutes.

`P:Task_Weaver_Backend.Models.DTOs.TaskChunkSettingsDto.MinMinutes`
Gets or sets the minimum chunk size in minutes.

Task_Weaver_Backend.Models.DTOs.TaskDateDto

Represents a scheduled execution window for a task.

`P:Task_Weaver_Backend.Models.DTOs.TaskDateDto.Date`
Gets or sets the calendar day for the scheduled block.

`P:Task_Weaver_Backend.Models.DTOs.TaskDateDto.End`
Gets or sets the end time of the scheduled block.

`P:Task_Weaver_Backend.Models.DTOs.TaskDateDto.IsManuallyMoved`
Marks chunks that were explicitly moved by drag-and-drop. Such chunks must not be overwritten by other manual moves.

`P:Task_Weaver_Backend.Models.DTOs.TaskDateDto.Start`
Gets or sets the start time of the scheduled block.

Task_Weaver_Backend.Models.DTOs.TaskDto

Represents a task with the planning inputs needed for scheduling and flow protection.

`P:Task_Weaver_Backend.Models.DTOs.TaskDto.CustomChunkSettings`
Gets or sets flow-protection chunk settings.

`P:Task_Weaver_Backend.Models.DTOs.TaskDto.Date`

Gets or sets the calendar day for the Task (null when type dynamic).

`P:Task_Weaver_Backend.Models.DTOs.TaskDto.Dates`

Gets or sets the generated task execution windows.

`P:Task_Weaver_Backend.Models.DTOs.TaskDto.Deadline`

Gets or sets the task deadline. (Null when Task type is not dynamic)

`P:Task_Weaver_Backend.Models.DTOs.TaskDto.Description`

Gets or sets the task description.

`P:Task_Weaver_Backend.Models.DTOs.TaskDto.End`

ISO 8601 end date/time. (null when type dynamic)

`P:Task_Weaver_Backend.Models.DTOs.TaskDto.EnergyLevel`

Gets or sets the cognitive energy level of the task.

`P:Task_Weaver_Backend.Models.DTOs.TaskDto.Id`

Gets or sets the Firestore document id.

`P:Task_Weaver_Backend.Models.DTOs.TaskDto.IsCompleted`

Gets or sets whether the task is completed.

`P:Task_Weaver_Backend.Models.DTOs.TaskDto.NeedToBeFinishedTaskIds`

Gets or sets the identifiers of predecessor tasks. (Null when Task type is not dynamic)

`P:Task_Weaver_Backend.Models.DTOs.TaskDto.Organization`

Gets or sets the organization/company assignment.

`P:Task_Weaver_Backend.Models.DTOs.TaskDto.PreferImmediateContinuation`

Optional patch-only hint: for running dynamic tasks, prefer immediate continuation of the currently active block instead of reordering by normal priority first. Not persisted to Firestore.

`P:Task_Weaver_Backend.Models.DTOs.TaskDto.Priority`

Gets or sets the task priority.

`P:Task_Weaver_Backend.Models.DTOs.TaskDto.RecurrenceCount`

Gets or sets the recurrence count for repeating tasks. (null when type dynamic)

`P:Task_Weaver_Backend.Models.DTOs.TaskDto.ReschedulePastChunks`

Optional patch-only scheduling flag: when

`P:Task_Weaver_Backend.Models.DTOs.TaskDto.Start`

ISO 8601 start date/time. (null when type dynamic)

`P:Task_Weaver_Backend.Models.DTOs.TaskDto.TimeEstimateMinutes`

Gets or sets the estimated effort in minutes.

`P:Task_Weaver_Backend.Models.DTOs.TaskDto.Title`

Gets or sets the task title.

`P:Task_Weaver_Backend.Models.DTOs.TaskDto.Type`

Gets or sets the task type.

Task_Weaver_Backend.Models.DTOs.UnresolvableTaskDto

Represents a task that could not be scheduled with the reason why.

`P:Task_Weaver_Backend.Models.DTOs.UnresolvableTaskDto.ErrorCode`

Gets or sets the error code for programmatic handling.

`P:Task_Weaver_Backend.Models.DTOs.UnresolvableTaskDto.Reason`

Gets or sets the human-readable reason why the task could not be scheduled.

`P:Task_Weaver_Backend.Models.DTos.UnresolvableTaskDto.TaskId`
Gets or sets the ID of the task that could not be scheduled.

Task_Weaver_Backend.Models.ErrorCodes

Central class for specific error code constants used by the frontend for mapping.

`F:Task_Weaver_Backend.Models.ErrorCodes.ChunkSettingsInvalid`
The chunk settings are invalid (e.g., min greater than max, or min less than 1).

`F:Task_Weaver_Backend.Models.ErrorCodes.ChunkSettingsNotFound`
The chunk settings were not found for the user.

`F:Task_Weaver_Backend.Models.ErrorCodes.InsufficientPermissions`
The user does not have sufficient permissions to perform this operation.

`F:Task_Weaver_Backend.Models.ErrorCodes.InviteExpired`
The invitation has expired and can no longer be accepted.

`F:Task_Weaver_Backend.Models.ErrorCodes.InviteNotFound`
The requested invitation was not found.

`F:Task_Weaver_Backend.Models.ErrorCodes.InviteToEmployee`
The user does not have sufficient permissions to perform this operation.

`F:Task_Weaver_Backend.Models.ErrorCodes.LastAdminProtection`
The operation is blocked because this user is the last admin of the organization.

`F:Task_Weaver_Backend.Models.ErrorCodes.MemberNotFound`
The requested member was not found in the organization.

`F:Task_Weaver_Backend.Models.ErrorCodes.OrganizationLastAdminCannotLeave`
The operation is blocked because this user is the last admin of the organization.

`F:Task_Weaver_Backend.Models.ErrorCodes.OrganizationMembershipNotFound`
The user is not a member of the requested organization.

`F:Task_Weaver_Backend.Models.ErrorCodes.OrganizationNotFound`
The requested organization was not found.

`F:Task_Weaver_Backend.Models.ErrorCodes.ScheduleCascadingDeadlineClash`
A cascading deadline clash was detected after static task fixation.

`F:Task_Weaver_Backend.Models.ErrorCodes.ScheduleChunkConstraintViolation`
A task's duration cannot be divided into valid chunks that satisfy `MinChunkSize` and `MaxChunkSize` constraints.

`F:Task_Weaver_Backend.Models.ErrorCodes.ScheduleCognitiveLimitExceeded`
A task cannot be scheduled because it exceeds the daily cognitive budget or real-time limit for its energy level.

`F:Task_Weaver_Backend.Models.ErrorCodes.ScheduleCyclicDependency`
A cyclic dependency was detected in the task graph.

`F:Task_Weaver_Backend.Models.ErrorCodes.ScheduleImpossible`
The scheduling algorithm could not find a valid schedule.

`F:Task_Weaver_Backend.Models.ErrorCodes.ScheduleInitialDeadlineViolation`
A task's deadline is already violated before scheduling begins.

`F:Task_Weaver_Backend.Models.ErrorCodes.ScheduleInvalidTimeFormat`
The time format is invalid. Expected "HH:mm" format.

`F:Task_Weaver_Backend.Models.ErrorCodes.ScheduleNotFound`

The requested working schedule was not found.

`F:Task_Weaver_Backend.Models.ErrorCodes.ScheduleOrphanDependency`
A task references a non-existent dependency.

`F:Task_Weaver_Backend.Models.ErrorCodes.ScheduleOutsideWorkHours`
A task is scheduled outside of the user's defined working hours.

`F:Task_Weaver_Backend.Models.ErrorCodes.ScheduleStartAfterEnd`
The start time must be before the end time in a schedule slot.

`F:Task_Weaver_Backend.Models.ErrorCodes.ScheduleStaticClash`
Static tasks overlap with each other or with blockers.

`F:Task_Weaver_Backend.Models.ErrorCodes.ScheduleTimeConflict`
A time slot conflicts with an existing schedule in the same week range.

`F:Task_Weaver_Backend.Models.ErrorCodes.ScheduleTimeParadox`
A time paradox was detected (e.g., sum of durations exceeds available time).

`F:Task_Weaver_Backend.Models.ErrorCodes.ScheduleTimeout`
The scheduling algorithm timed out.

`F:Task_Weaver_Backend.Models.ErrorCodes.UserAlreadyMember`
The user is already a member of this organization.

`F:Task_Weaver_Backend.Models.ErrorCodes.ValidationError`
Generic validation error for invalid field data.

`F:Task_Weaver_Backend.Models.ErrorCodes.WhitelistConflict`
The domain or email is already whitelisted by another organization.

Task_Weaver_Backend.Models.ErrorDetailDto

DTO representing a specific field error.

`P:Task_Weaver_Backend.Models.ErrorDetailDto.Field`
The name of the field that caused the error.

`P:Task_Weaver_Backend.Models.ErrorDetailDto.Issue`
A description of the issue with the field.

Task_Weaver_Backend.Models.ErrorDetailsDto

Represents the error details dto.

Task_Weaver_Backend.Models.ErrorResponseDto

DTO representing a structured error response.

`P:Task_Weaver_Backend.Models.ErrorResponseDto.Details`
Optional field-specific error details or a general reason string.

`P:Task_Weaver_Backend.Models.ErrorResponseDto.Message`
Deprecated user-facing message field kept temporarily for compatibility. Hidden from JSON contract in errorCode-only mode.

Task_Weaver_Backend.Models.GlobalChunkSettingsDto

Data Transfer Object representing global chunk size settings for a user. Defines the default minimum and maximum duration for continuous work phases.

`P:Task_Weaver_Backend.Models.GlobalChunkSettingsDto.DefaultMaxChunkMinutes`

The default maximum chunk duration in minutes. Must be greater than or equal to

`P:Task_Weaver_Backend.Models.GlobalChunkSettingsDto.DefaultMinChunkMinutes`
The default minimum chunk duration in minutes. Must be at least 1 minute.

Task_Weaver_Backend.Models.IBlockerRepository

Repository interface for calendar blocker operations.

`M:Task_Weaver_Backend.Models.IBlockerRepository.CreateCalendarBlockerAsync(System.String,Task_Weaver_Backend.Models.BlockerDto)`
Creates a new calendar blocker for a user.

Parameter

- `uid` : The user's Firebase UID.
- `blocker` : The blocker data to create.

`M:Task_Weaver_Backend.Models.IBlockerRepository.DeleteCalendarBlockerAsync(System.String,System.String)`
Deletes a calendar blocker.

Parameter

- `uid` : The user's Firebase UID.
- `blockerId` : The blocker's document ID.

`M:Task_Weaver_Backend.Models.IBlockerRepository.GetCalendarBlockersAsync(System.String)`
Retrieves all calendar blockers for a specific user.

Parameter

- `uid` : The user's Firebase UID.

Returns: A list of all blockers for the user.

`M:Task_Weaver_Backend.Models.IBlockerRepository.PatchCalendarBlockerAsync(System.String,System.String,Task_Weaver_Backend.Models.BlockerDto)`
Updates an existing calendar blocker.

Parameter

- `uid` : The user's Firebase UID.
- `blockerId` : The blocker's document ID.
- `blocker` : The updated blocker data.

Task_Weaver_Backend.Models.IOrganizationRepository

Repository contract for organization persistence operations.

`M:Task_Weaver_Backend.Models.IOrganizationRepository.AddEmployeeToOrganizationAsync(System.String,System.String)`
Adds one user ID to the organization member array.

Parameter

- `orgId` : Organization identifier.
- `uid` : User UID to add.

`M:Task_Weaver_Backend.Models.IOrganizationRepository.CreateOrganizationInviteAsync(System.String,Task_Weaver_Backend.Models.InviteDto)`
Creates a new invitation document in the organization's INVITES subcollection.

Parameter

- `orgId` : Organization identifier.
- `invite` : Invitation payload.

Returns: The generated invitation identifier.

`M:Task_Weaver_Backend.Models.IOrganizationRepository.DeleteOrganizationInviteAsync(System.String,System.String)`
Deletes an invitation from the organization's INVITES subcollection.

Parameter

- `orgId` : Organization identifier.
- `inviteId` : Invitation identifier.

`M:Task_Weaver_Backend.Models.IOrganizationRepository.GetOrganizationByIdAsync(System.String)`
Loads an organization by document identifier.

Parameter

- `orgId` : Organization identifier.

Returns: The organization or

`M:Task_Weaver_Backend.Models.IOrganizationRepository.GetOrganizationByInviteIdAsync(System.String)`
Loads an organization by invite identifier.

Parameter

- `inviteId` : Invite identifier.

Returns: The organization or

M:Task_Weaver_Backend.Models.IOrganizationRepository.GetOrganizationIdsByUidAsync(System.String)
Retrieves all organization document identifiers where the user is listed as a member.

Parameter

- `uid` : User UID.

Returns: List of organization identifiers.

M:Task_Weaver_Backend.Models.IOrganizationRepository.GetOrganizationInviteByIdAsync(System.String,System.String)
Retrieves an invitation by its identifier.

Parameter

- `orgId` : Organization identifier.
- `inviteId` : Invitation identifier.

Returns: The invitation or

M:Task_Weaver_Backend.Models.IOrganizationRepository.GetOrganizationInvites(System.String)
Retrieves all pending invitations for an organization.

Parameter

- `orgId` : Organization identifier.

Returns: List of pending invitations.

M:Task_Weaver_Backend.Models.IOrganizationRepository.GetOrganizationSettingsAsync(System.String)
Retrieves organization settings.

Parameter

- `orgId` : Organization identifier.

Returns: The organization settings or

M:Task_Weaver_Backend.Models.IOrganizationRepository.RemoveEmployeeFromOrganizationAsync(System.String,System.String)
Removes one user ID from the organization members array.

Parameter

- `orgId` : Organization identifier.
- `uid` : User UID to remove.

M:Task_Weaver_Backend.Models.IOrganizationRepository.UpdateOrganizationMemberRole(System.String,System.String,System.Boolean)
Adds or removes one user ID from the organization admins array.

Parameter

- `orgId` : Organization identifier.
- `uid` : User UID to add.
- `isAdmin` :

M:Task_Weaver_Backend.Models.IOrganizationRepository.UpdateOrganizationSettingsAsync(System.String,Task_Weaver_Backend.Models.OrganizationSettingsDto)
Updates organization settings.

Parameter

- `orgId` : Organization identifier.
- `settings` : Settings payload.

Task_Weaver_Backend.Models.ITaskRepository

interface ITaskRepository.

M:Task_Weaver_Backend.Models.ITaskRepository.CreateTaskAsync(System.String,Task_Weaver_Backend.Models.DTOS.TaskDto)
Creates a new calendar task for a user.

Parameter

- `uid` : The user's Firebase UID.
- `task` : The task to create.

Returns: The created task including its generated ID.

M:Task_Weaver_Backend.Models.ITaskRepository.DeleteTaskAsync(System.String,System.String)
Deletes a calendar task.

Parameter

- `uid` : The user's Firebase UID.
- `taskId` : The task document ID.

M:Task_Weaver_Backend.Models.ITaskRepository.GetTaskAsync(System.String,System.String)
Retrieves a single calendar task by its identifier.

Parameter

- `uid` : The user's Firebase UID.
- `taskId` : The task document ID.

Returns: The task or

M:Task_Weaver_Backend.Models.ITaskRepository.GetTasksAsync(System.String)
Retrieves all calendar tasks for a user.

Parameter

- `uid` : The user's Firebase UID.

Returns: A list of calendar tasks.

`M:Task_Weaver_Backend.Models.ITaskRepository.UpdateTaskAsync(System.String,System.String,Task_Weaver_Backend.Models.DTos.TaskDto)`
Updates selected fields of an existing calendar task.

Parameter

- `uid` : The user's Firebase UID.
- `taskId` : The task document ID.
- `taskUpdate` : The partial set of fields to update.

Task_Weaver_Backend.Models.IUserRepository

Repository contract for user profile and user-scoped resources.

`M:Task_Weaver_Backend.Models.IUserRepository.CreateUserAsync(System.String,Task_Weaver_Backend.Models.UserProfileDto)`
Creates a user profile document.

Parameter

- `uid` : Firebase user identifier.
- `profile` : Profile payload.

`M:Task_Weaver_Backend.Models.IUserRepository.DeleteUserAsync(System.String)`
Deletes a user profile document.

Parameter

- `uid` : Firebase user identifier.

`M:Task_Weaver_Backend.Models.IUserRepository.GetByIdAsync(System.String)`
Loads a user profile by UID.

Parameter

- `uid` : Firebase user identifier.

Returns: User profile or

`M:Task_Weaver_Backend.Models.IUserRepository.GetByLoginEmailAsync(System.String)`
Loads a user profile by login email.

Parameter

- `loginEmail` : User login email.

Returns: User profile or

`M:Task_Weaver_Backend.Models.IUserRepository.UpdateAsync(System.String,Task_Weaver_Backend.Models.UserProfileDto)`
Updates profile fields on an existing user.

Parameter

- `uid` : Firebase user identifier.
- `updateData` : Patch payload.

Task_Weaver_Backend.Models.IWorkProfileRepository

Repository interface for working schedule operations.

`M:Task_Weaver_Backend.Models.IWorkProfileRepository.CreateWorkingScheduleAsync(System.String,Task_Weaver_Backend.Models.WorkingScheduleDto)`
Creates a new working schedule for a user.

Parameter

- `uid` : The user's Firebase UID.
- `schedule` : The schedule data to create.

Returns: The created schedule with its generated ID.

`M:Task_Weaver_Backend.Models.IWorkProfileRepository.DeleteWorkingScheduleAsync(System.String,System.String)`
Deletes a working schedule.

Parameter

- `uid` : The user's Firebase UID.
- `scheduleId` : The schedule's document ID.

`M:Task_Weaver_Backend.Models.IWorkProfileRepository.GetGlobalChunkSettingsAsync(System.String)`
Retrieves the global chunk settings for a user.

Parameter

- `uid` : The user's Firebase UID.

Returns: The user's global chunk settings, or null if not set.

`M:Task_Weaver_Backend.Models.IWorkProfileRepository.GetWorkingSchedulesAsync(System.String)`
Retrieves all working schedules for a specific user.

Parameter

- `uid` : The user's Firebase UID.

Returns: A list of all working schedules for the user.

`M:Task_Weaver_Backend.Models.IWorkProfileRepository.PatchWorkingScheduleAsync(System.String,System.String,Task_Weaver_Backend.Models.WorkingSchedule)`
Updates an existing working schedule.

Parameter

- `uid` : The user's Firebase UID.
- `scheduleId` : The schedule's document ID.
- `schedule` : The updated schedule data.

Returns: The updated schedule.

`M:Task_Weaver_Backend.Models.IWorkProfileRepository.UpdateGlobalChunkSettingsAsync(System.String,Task_Weaver_Backend.Models.GlobalChunkSettingsDto)`
Updates the global chunk settings for a user.

Parameter

- `uid` : The user's Firebase UID.
- `settings` : The new chunk settings.

Task_Weaver_Backend.Models.InviteDto

Data Transfer Object representing an invitation to join an organization.

`P:Task_Weaver_Backend.Models.InviteDto.Email`
Gets or sets the email address of the invited user.

`P:Task_Weaver_Backend.Models.InviteDto.ExpiresAt`
Gets or sets the expiration timestamp for the invitation.

`P:Task_Weaver_Backend.Models.InviteDto.Id`
Gets or sets the unique identifier for the invitation.

Task_Weaver_Backend.Models.MemberDto

Data Transfer Object representing a member of an organization with role information.

`P:Task_Weaver_Backend.Models.MemberDto.Email`
Gets or sets the email address of the member.

`P:Task_Weaver_Backend.Models.MemberDto.FirstName`
Gets or sets the first name of the member.

`P:Task_Weaver_Backend.Models.MemberDto.IsAdmin`
Gets or sets a value indicating whether the member has admin privileges.

`P:Task_Weaver_Backend.Models.MemberDto.LastName`
Gets or sets the last name of the member.

`P:Task_Weaver_Backend.Models.MemberDto.SystemRole`
Gets or sets the system Role of the member.

`P:Task_Weaver_Backend.Models.MemberDto.Uid`
Gets or sets the unique identifier of the user.

Task_Weaver_Backend.Models.OrganizationDto

Data Transfer Object representing an Organization in the system.

Task_Weaver_Backend.Models.OrganizationRepository

Firestore-backed repository for organization documents.

`M:Task_Weaver_Backend.Models.OrganizationRepository.#ctor(Google.Cloud.Firestore.FirestoreDb)`
Initializes a new instance of the

Parameter

- `db` : The db.

`M:Task_Weaver_Backend.Models.OrganizationRepository.AddEmployeeToOrganizationAsync(System.String,System.String)`

Executes add employee to organization async.

Parameter

- `orgId` : The organization identifier.
- `uid` : The user identifier.

Returns: A task that represents the asynchronous operation.

`M:Task_Weaver_Backend.Models.OrganizationRepository.CreateOrganizationInviteAsync(System.String,Task_Weaver_Backend.Models.InviteDto)`

Performs create organization invite async.

Parameter

- `orgId` : The organization identifier.
- `invite` : The invite.

Returns: A result indicating successful creation.

`M:Task_Weaver_Backend.Models.OrganizationRepository.DeleteOrganizationInviteAsync(System.String,System.String)`

Executes delete organization invite async.

Parameter

- `orgId` : The organization identifier.
- `inviteId` : The invite identifier.

Returns: A task that represents the asynchronous operation.

`M:Task_Weaver_Backend.Models.OrganizationRepository.GetAllDomainWhitelistAsync`

Performs get all domain whitelist async.

Returns: The requested all domain whitelist async.

`M:Task_Weaver_Backend.Models.OrganizationRepository.GetAllEmailWhitelistAsync`

Performs get all email whitelist async.

Returns: The requested all email whitelist async.

`M:Task_Weaver_Backend.Models.OrganizationRepository.GetOrgIdFromWhitelistedEmail(System.String)`

Performs get org id from whitelisted email.

Parameter

- `email` : The email address.

Returns: The requested org id from whitelisted email.

`M:Task_Weaver_Backend.Models.OrganizationRepository.GetOrganizationByIdAsync(System.String)`

Executes get organization by id async.

Parameter

- `orgId` : The organization identifier.

Returns: A task that resolves to the requested result.

`M:Task_Weaver_Backend.Models.OrganizationRepository.GetOrganizationByInviteIdAsync(System.String)`

Executes get organization by invite id async.

Parameter

- `inviteId` : The invite identifier.

Returns: A task that resolves to the requested result.

`M:Task_Weaver_Backend.Models.OrganizationRepository.GetOrganizationIdsByUidAsync(System.String)`

Performs get organization ids by uid async.

Parameter

- `uid` : The user identifier.

Returns: The requested organization ids by uid async.

`M:Task_Weaver_Backend.Models.OrganizationRepository.GetOrganizationInviteByIdAsync(System.String,System.String)`

Performs get organization invite by id async.

Parameter

- `orgId` : The organization identifier.
- `inviteId` : The invite identifier.

Returns: The requested organization invite by id async.

`M:Task_Weaver_Backend.Models.OrganizationRepository.GetOrganizationInvites(System.String)`

Performs get organization invites.

Parameter

- `orgId` : The organization identifier.

Returns: The requested organization invites.

`M:Task_Weaver_Backend.Models.OrganizationRepository.GetOrganizationSettingsAsync(System.String)`

Executes get organization settings async.

Parameter

- `orgId` : The organization identifier.

Returns: A task that resolves to the requested result.

`M:Task_Weaver_Backend.Models.OrganizationRepository.RemoveEmployeeFromOrganizationAsync(System.String,System.String)`

Executes remove employee from organization async.

Parameter

- `orgId` : The organization identifier.
- `uid` : The user identifier.

Returns: A task that represents the asynchronous operation.

`M:Task_Weaver_Backend.Models.OrganizationRepository.UpdateOrganizationMemberRole(System.String,System.String,System.Boolean)`

Performs update organization member role.

Parameter

- `orgId` : The organization identifier.
- `uid` : The user identifier.
- `isAdmin` : The is admin.

Returns: A task that represents the asynchronous operation.

`M:Task_Weaver_Backend.Models.OrganizationRepository.UpdateOrganizationSettingsAsync(System.String,Task_Weaver_Backend.Models.OrganizationSettingsDto)`

Executes update organization settings async.

Parameter

- `orgId` : The organization identifier.
- `settings` : The settings.

Returns: A task that represents the asynchronous operation.

Task_Weaver_Backend.Models.OrganizationRoleDto

Represents a user's role metadata within a specific organization.

`P:Task_Weaver_Backend.Models.OrganizationRoleDto.IsAdmin`

Indicates whether the user has administrator privileges in the organization.

`P:Task_Weaver_Backend.Models.OrganizationRoleDto.OrganizationId`

The identifier of the related organization.

Task_Weaver_Backend.Models.OrganizationSettingsDto

Data Transfer Object for organization-specific settings.

`P:Task_Weaver_Backend.Models.OrganizationSettingsDto.DomainWhiteList`

Gets or sets the list of whitelisted email domains (e.g., @company.com).

`P:Task_Weaver_Backend.Models.OrganizationSettingsDto.EmailWhiteList`

Gets or sets the list of whitelisted specific email addresses.

`P:Task_Weaver_Backend.Models.OrganizationSettingsDto.InviteLinkValidityDays`

Gets or sets the number of days an invitation link remains valid. Default is 30 days.

Task_Weaver_Backend.Models.ScheduleDayDto

Data Transfer Object representing a single day's working time slot within a schedule.

`P:Task_Weaver_Backend.Models.ScheduleDayDto.End`

The end time of the working slot in "HH:mm" format (e.g., "17:00"). Must be after

`P:Task_Weaver_Backend.Models.ScheduleDayDto.OrganizationId`

Optional organization ID for exclusive time slots. When set, the scheduler only assigns tasks from this specific organization. When null or empty, this is a "Flex-Block" allowing tasks from any linked organization.

`P:Task_Weaver_Backend.Models.ScheduleDayDto.Start`

The start time of the working slot in "HH:mm" format (e.g., "08:00").

Task_Weaver_Backend.Models.TaskRepository

Represents the task repository.

`M:Task_Weaver_Backend.Models.TaskRepository.#ctor(Google.Cloud.Firestore.FirestoreDb)`

Performs task repository.

Parameter

- `db` : The db.

`M:Task_Weaver_Backend.Models.TaskRepository.CreateTaskAsync(System.String,Task_Weaver_Backend.Models.DT0s.TaskDto)`

Performs create task async.

Parameter

- `uid` : The user identifier.
- `task` : The task.

Returns: A task that represents the asynchronous operation.

`M:Task_Weaver_Backend.Models.TaskRepository.DeleteTaskAsync(System.String,System.String)`

Performs delete task async.

Parameter

- `uid` : The user identifier.
- `taskId` : The task identifier.

Returns: A task that represents the asynchronous operation.

`M:Task_Weaver_Backend.Models.TaskRepository.GetTaskAsync(System.String,System.String)`

Performs get task async.

Parameter

- `uid` : The user identifier.
- `taskId` : The task identifier.

Returns: The requested task async.

`M:Task_Weaver_Backend.Models.TaskRepository.GetTasksAsync(System.String)`

Performs get tasks async.

Parameter

- `uid` : The user identifier.

Returns: The requested tasks async.

`M:Task_Weaver_Backend.Models.TaskRepository.UpdateTaskAsync(System.String,System.String,Task_Weaver_Backend.Models.DT0s.TaskDto)`

Performs update task async.

Parameter

- `uid` : The user identifier.
- `taskId` : The task identifier.
- `taskUpdate` : The task update.

Returns: A task that represents the asynchronous operation.

Task_Weaver_Backend.Models.UserOrganizationDto

Data Transfer Object for listing organizations of the authenticated user. Includes organization data and the user's role metadata within that organization.

`P:Task_Weaver_Backend.Models.UserOrganizationDto.Id`

The unique organization identifier.

`P:Task_Weaver_Backend.Models.UserOrganizationDto.Name`

The organization display name.

Task_Weaver_Backend.Models.UserRepository

Firestore-backed repository for user profiles and user-scoped resources.

`M:Task_Weaver_Backend.Models.UserRepository.#ctor(Google.Cloud.Firestore.FirestoreDb)`

Initializes a new instance of the

Parameter

- `db` : The db.

`M:Task_Weaver_Backend.Models.UserRepository.CreateUserAsync(System.String,Task_Weaver_Backend.Models.UserProfileDto)`

Executes create user async.

Parameter

- `uid` : The user identifier.
- `profile` : The profile.

Returns: A task that represents the asynchronous operation.

`M:Task_Weaver_Backend.Models.UserRepository.DeleteUserAsync(System.String)`
Performs delete user async.

Parameter

- `uid` : The user identifier.

Returns: A task that represents the asynchronous operation.

`M:Task_Weaver_Backend.Models.UserRepository.GetByIdAsync(System.String)`
Executes get by id async.

Parameter

- `uid` : The user identifier.

Returns: A task that resolves to the requested result.

`M:Task_Weaver_Backend.Models.UserRepository.GetByLoginEmailAsync(System.String)`
Executes get by login email async.

Parameter

- `loginEmail` : The login email.

Returns: A task that resolves to the requested result.

`M:Task_Weaver_Backend.Models.UserRepository.UpdateAsync(System.String,Task_Weaver_Backend.Models.UserProfileDto)`
Executes update async.

Parameter

- `uid` : The user identifier.
- `updateData` : The update data.

Returns: A task that represents the asynchronous operation.

Task_Weaver_Backend.Models.WhitelistRegistryDto

Data Transfer Object for the global whitelist registry to enforce uniqueness. Each domain or email can only be claimed by one organization.

`P:Task_Weaver_Backend.Models.WhitelistRegistryDto.ClaimedAt`
Gets or sets the timestamp when this entry was claimed.

`P:Task_Weaver_Backend.Models.WhitelistRegistryDto.Id`
Gets or sets the unique identifier (domain or email address).

`P:Task_Weaver_Backend.Models.WhitelistRegistryDto.OrganizationId`
Gets or sets the organization ID that has claimed this whitelist entry.

`P:Task_Weaver_Backend.Models.WhitelistRegistryDto.Type`
Gets or sets the type of whitelist entry ("domain" or "email").

Task_Weaver_Backend.Models.WorkProfileRepository

Firestore-backed repository for working schedule operations.

`M:Task_Weaver_Backend.Models.WorkProfileRepository.#ctor(Google.Cloud.Firestore.FirestoreDb)`
Initializes a new instance of the

Parameter

- `db` : The db.

`M:Task_Weaver_Backend.Models.WorkProfileRepository.CreateWorkingScheduleAsync(System.String,Task_Weaver_Backend.Models.WorkingScheduleDto)`
Executes create working schedule async.

Parameter

- `uid` : The user identifier.
- `schedule` : The schedule.

Returns: A task that resolves to the requested result.

`M:Task_Weaver_Backend.Models.WorkProfileRepository.DeleteWorkingScheduleAsync(System.String,System.String)`
Executes delete working schedule async.

Parameter

- `uid` : The user identifier.
- `scheduleId` : The schedule identifier.

Returns: A task that represents the asynchronous operation.

`M:Task_Weaver_Backend.Models.WorkProfileRepository.GetGlobalChunkSettingsAsync(System.String)`
Executes get global chunk settings async.

Parameter

- `uid` : The user identifier.

Returns: A task that resolves to the requested result.

`M:Task_Weaver_Backend.Models.WorkProfileRepository.GetWorkingScheduleAsync(System.String,System.String)`

Performs get working schedule async.

Parameter

- `uid` : The user identifier.
- `workingScheduleId` : The working schedule id.

Returns: The requested working schedule async.

`M:Task_Weaver_Backend.Models.WorkProfileRepository.GetWorkingSchedulesAsync(System.String)`

Executes get working schedules async.

Parameter

- `uid` : The user identifier.

Returns: A task that resolves to the requested result.

`M:Task_Weaver_Backend.Models.WorkProfileRepository.PatchWorkingScheduleAsync(System.String,System.String,Task_Weaver_Backend.Models.WorkingScheduleD`

Executes patch working schedule async.

Parameter

- `uid` : The user identifier.
- `scheduleId` : The schedule identifier.
- `schedule` : The schedule.

Returns: A task that resolves to the requested result.

`M:Task_Weaver_Backend.Models.WorkProfileRepository.UpdateGlobalChunkSettingsAsync(System.String,Task_Weaver_Backend.Models.GlobalChunkSettingsDto)`

Executes update global chunk settings async.

Parameter

- `uid` : The user identifier.
- `settings` : The settings.

Returns: A task that represents the asynchronous operation.

Task_Weaver_Backend.Models.WorkingScheduleDto

Data Transfer Object representing a working schedule for a specific week range. Contains availability time slots organized by weekday.

`P:Task_Weaver_Backend.Models.WorkingScheduleDto.Id`

The unique identifier of the working schedule.

`P:Task_Weaver_Backend.Models.WorkingScheduleDto.Schedule`

Dictionary mapping weekday names (lowercase, e.g., "monday", "tuesday") to their time slots. Each day can have multiple time slots defined in

`P:Task_Weaver_Backend.Models.WorkingScheduleDto.WeekEnd`

The ending week number (ISO 8601 week numbering, 1-53). Must be greater than or equal to

`P:Task_Weaver_Backend.Models.WorkingScheduleDto.WeekStart`

The starting week number (ISO 8601 week numbering, 1-53).

`P:Task_Weaver_Backend.Models.WorkingScheduleDto.Year`

The calendar year this schedule applies to (e.g., 2026).

Task_Weaver_Backend.Services.Algorithm.BacktrackingStack

Manages the backtracking stack for undo operations during scheduling.

`P:Task_Weaver_Backend.Services.Algorithm.BacktrackingStack.Count`

Gets the current stack size.

`M:Task_Weaver_Backend.Services.Algorithm.BacktrackingStack.IsEmpty`

Checks if the stack is empty.

Returns: True when the condition is met; otherwise false.

`M:Task_Weaver_Backend.Services.Algorithm.BacktrackingStack.Pop`

Pops the last placement entry from the stack.

Returns: The computed result.

`M:Task_Weaver_Backend.Services.Algorithm.BacktrackingStack.Push(Task_Weaver_Backend.Services.Algorithm.Models.BacktrackingEntry)`

Pushes a placement entry onto the stack.

Parameter

- `entry` : The entry.

Task_Weaver_Backend.Services.Algorithm.CalendarGrid

Calendar grid for efficient slot management with 15-minute granularity.

M:Task_Weaver_Backend.Services.Algorithm.CalendarGrid.#ctor(System.TimeSpan)

Performs calendar grid.

Parameter

- `slotSize` : The slot size.

M:Task_Weaver_Backend.Services.Algorithm.CalendarGrid.ClipAtMidnight(System.DateTime,System.TimeSpan)

Clips a duration at midnight (for multi-day task handling).

Parameter

- `slotStart` : The slot start.
- `duration` : The duration.

Returns: The computed result.

M:Task_Weaver_Backend.Services.Algorithm.CalendarGrid.ClipToWorkPlan(System.DateTime,System.DateTime)

Clips a time range to the available work slots in the grid. If the range is entirely outside the work plan, returns null.

Parameter

- `start` : The start.
- `end` : The end.

Returns: The clipped start/end range aligned to available slots, or

M:Task_Weaver_Backend.Services.Algorithm.CalendarGrid.FreeBlock(Task_Weaver_Backend.Services.Algorithm.Models.TimeBlock)

Frees a time block (for backtracking).

Parameter

- `block` : The block.

M:Task_Weaver_Backend.Services.Algorithm.CalendarGrid.GetAvailableDuration(System.DateTime)

Gets the available duration from a slot until the next blocked slot or end of day.

Parameter

- `slotStart` : The slot start.

Returns: The computed result.

M:Task_Weaver_Backend.Services.Algorithm.CalendarGrid.GetNextFreeSlot(System.DateTime)

Gets the next free slot starting from a given time.

Parameter

- `from` : The from.

Returns: The computed result.

M:Task_Weaver_Backend.Services.Algorithm.CalendarGrid.GetTotalFreeSlotsCount

Performs get total free slots count.

Returns: The requested total free slots count.

M:Task_Weaver_Backend.Services.Algorithm.CalendarGrid.Initialize(Task_Weaver_Backend.Services.Algorithm.Models.WorkPlan,Task_Weaver_Backend.Services)

Initializes the calendar grid from work plan and merged blocked zones.

Parameter

- `workPlan` : The work plan.
- `mergedBlockers` : The merged blockers.
- `startDate` : The start date.
- `endDate` : The end date.

M:Task_Weaver_Backend.Services.Algorithm.CalendarGrid.IsSlotFree(System.DateTime)

Checks if a specific slot is free.

Parameter

- `slot` : The slot.

Returns: True when the condition is met; otherwise false.

M:Task_Weaver_Backend.Services.Algorithm.CalendarGrid.MarkBlocked(Task_Weaver_Backend.Services.Algorithm.Models.TimeBlock)

Marks a time block as blocked (unavailable).

Parameter

- `block` : The block.

Task_Weaver_Backend.Services.Algorithm.ChunkEvaluator

Evaluates chunk sizes for tasks, resolving local vs. global settings.

M:Task_Weaver_Backend.Services.Algorithm.ChunkEvaluator.CalculateChunkDuration(System.TimeSpan,System.TimeSpan,System.TimeSpan,System.TimeSpan)

Calculates the chunk duration for a task placement. Enforces hard MinChunk constraint: returns TimeSpan.Zero if slot is too small.

Parameter

- slotDuration : The slot duration.
- maxChunk : The max chunk.
- minChunk : The min chunk.
- remainingDuration : The remaining duration.

Returns: The computed result.

M:Task_Weaver_Backend.Services.Algorithm.ChunkEvaluator.IsSlotTooSmall(System.TimeSpan,System.TimeSpan)

Checks if a slot is too small to accommodate the minimum chunk size.

Parameter

- slotDuration : The slot duration.
- underLimit : The under limit.

Returns: True when the condition is met; otherwise false.

M:Task_Weaver_Backend.Services.Algorithm.ChunkEvaluator.ResolveMaxChunkSize(Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask,Task_Weaver_Backend.Services.Algorithm.Models.GlobalSettings)

Resolves the maximum chunk size for a task (local override or global default).

Parameter

- task : The task.
- globalSettings : The global settings.

Returns: The computed result.

M:Task_Weaver_Backend.Services.Algorithm.ChunkEvaluator.ResolveMinChunkSize(Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask,Task_Weaver_Backend.Services.Algorithm.Models.GlobalSettings)

Resolves the minimum chunk size for a task (local override or global default).

Parameter

- task : The task.
- globalSettings : The global settings.

Returns: The computed result.

M:Task_Weaver_Backend.Services.Algorithm.ChunkEvaluator.ValidateTaskChunkCompatibility(System.TimeSpan,System.TimeSpan,System.TimeSpan)

Validates whether a task's duration can be divided into valid chunks that satisfy MinChunk and MaxChunk constraints. Returns true if the duration is compatible, false otherwise.

Parameter

- remainingDuration : The remaining duration.
- minChunk : The min chunk.
- maxChunk : The max chunk.

Returns: True when the condition is met; otherwise false.

Task_Weaver_Backend.Services.Algorithm.CognitiveGuard

Cognitive guard implementing a dual-tracker system for cognitive budget management. Prevents burnout by tracking both energy pool (weighted) and real-time limits per level.

M:Task_Weaver_Backend.Services.Algorithm.CognitiveGuard.CalculateBudgetCost(Task_Weaver_Backend.Services.Algorithm.Models.CognitiveLevel,System.TimeSpan)

Calculates the energy cost for a task chunk (Konto A). Level 3: Duration × 2, Level 2: Duration × 1, Level 1: 0.

Parameter

- level : The level.
- chunkDuration : The chunk duration.

Returns: The computed result.

M:Task_Weaver_Backend.Services.Algorithm.CognitiveGuard.DeductBudget(System.DateOnly,Task_Weaver_Backend.Services.Algorithm.Models.CognitiveLevel,System.TimeSpan)

Deducts budget from both accounts (energy pool and real-time tracker).

Parameter

- day : The day.
- level : The level.
- chunkDuration : The chunk duration.

M:Task_Weaver_Backend.Services.Algorithm.CognitiveGuard.GetEnergyBudgetForDay(System.DateOnly)

Gets the remaining energy budget for a specific day.

Parameter

- day : The day.

Returns: The computed result.

M:Task_Weaver_Backend.Services.Algorithm.CognitiveGuard.GetLevel2RealTimeBudgetForDay(System.DateOnly)

Gets the remaining Level 2 real-time budget for a specific day.

Parameter

- day : The day.

Returns: The computed result.

Gets the remaining Level 3 real-time budget for a specific day.

- `day` : The day.

Gets the hard real-time limit for a specific cognitive level.

- `level`: The level.

- `day` : The day.
- `level` : The level.

- `day` : The day.
- `level` : The level.
- `chunkDuration` : The chunk duration.

- `workDays` : The work days.

- **day** : The day.

- **day** : The day.

- **day** : The day.

- `level`: The level.

- `day` : The day.
- `level` : The level.
- `chunkDuration` : The chunk duration.

```
M:Task_Weaver_Backend.Services.Algorithm.CognitiveGuard.SetCooldownStatus(System.Boolean)
```

Sets the cooldown status.

Parameter

- `status` : The status.

```
M:Task_Weaver_Backend.Services.Algorithm.CognitiveGuard.SetDailyCognitiveLimit(System.TimeSpan)
```

Configure the daily cognitive limit (for tests/configuration).

Parameter

- `limit` : The limit.

Task_Weaver_Backend.Services.Algorithm.CpmCalculator

Critical Path Method (CPM) calculator for deadline analysis and slack computation.

```
M:Task_Weaver_Backend.Services.Algorithm.CpmCalculator.Calculate(Task_Weaver_Backend.Services.Algorithm.Models.DependencyGraph,System.Collections.Generic.IEnumerable{Task_Weaver_Backend.Services.Algorithm.Models.Task})
```

Calculates CPM metrics (Earliest Start, Latest Finish, Slack) for all tasks. Note: For most accurate results, tasks should be topologically sorted before calling this method. However, the algorithm is designed to handle unsorted lists by iterating through dependencies.

Parameter

- `graph` : The graph.
- `tasks` : The tasks.
- `existingBlocks` : The existing blocks.
- `now` : The now.

Returns: The resulting collection.

```
M:Task_Weaver_Backend.Services.Algorithm.CpmCalculator.DetectTimeParadoxes(System.Collections.Generic.IEnumerable{Task_Weaver_Backend.Services.Algorithm.Models.Task},System.Collections.Generic.IEnumerable{Task_Weaver_Backend.Services.Algorithm.Models.Task})
```

Detects time paradoxes (e.g., sum of task durations exceeds available time window).

Parameter

- `results` : The results.
- `tasks` : The tasks.
- `now` : The now.
- `userTimeZone` : The user time zone.

```
M:Task_Weaver_Backend.Services.Algorithm.CpmCalculator.HasNegativeSlack(System.Collections.Generic.IEnumerable{Task_Weaver_Backend.Services.Algorithm.Models.Task})
```

Checks if any task has negative slack (deadline violation).

Parameter

- `results` : The results.

Returns: True when the condition is met; otherwise false.

```
M:Task_Weaver_Backend.Services.Algorithm.CpmCalculator.Reevaluate(Task_Weaver_Backend.Services.Algorithm.Models.DependencyGraph,System.Collections.Generic.IEnumerable{Task_Weaver_Backend.Services.Algorithm.Models.Task})
```

Re-evaluates CPM after static task fixation.

Parameter

- `graph` : The graph.
- `dynamicTasks` : The dynamic tasks.
- `existingBlocks` : The existing blocks.
- `now` : The now.

Returns: The resulting collection.

```
M:Task_Weaver_Backend.Services.Algorithm.CpmCalculator.ValidateAsapSafety(Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask,System.Collections.Generic.IEnumerable{Task_Weaver_Backend.Services.Algorithm.Models.Task})
```

Validates ASAP safety: checks if an ASAP task would push other tasks past their deadlines.

Parameter

- `asapTask` : The asap task.
- `otherTasks` : The other tasks.
- `cpmResults` : The cpm results.

Returns: True when the condition is met; otherwise false.

Task_Weaver_Backend.Services.Algorithm.DagBuilder

Builds and validates a directed acyclic graph (DAG) from task dependencies.

```
M:Task_Weaver_Backend.Services.Algorithm.DagBuilder.Build(System.Collections.Generic.IEnumerable{Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask})
```

Builds a dependency graph from a list of tasks.

Parameter

- `tasks` : The tasks.

Returns: The computed result.

```
M:Task_Weaver_Backend.Services.Algorithm.DagBuilder.CheckOrphans(System.Collections.Generic.IEnumerable{Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask})
```

Checks for orphan dependencies (references to non-existent tasks). Throws immediately on the first missing dependency.

Parameter

- `tasksToCheck` : The tasks to check.
- `allTasks` : The all tasks.

Returns: True when the condition is met; otherwise false.

`M:Task_Weaver_Backend.Services.Algorithm.DagBuilder.DetectCycles(Task_Weaver_Backend.Services.Algorithm.Models.DependencyGraph)`

Detects cycles in the dependency graph using DFS.

Parameter

- `graph` : The graph.

Returns: True when the condition is met; otherwise false.

`M:Task_Weaver_Backend.Services.Algorithm.DagBuilder.TopologicalSort(Task_Weaver_Backend.Services.Algorithm.Models.DependencyGraph,System.Collections.Generic.IEnumerable<Task_Weaver_Backend.Services.Algorithm.Models.Task>)`
Performs topological sort with priority ordering. Sorting criteria: 1. In-Degree = 0, 2. Deadline (asc), 3. Priority (desc), 4. ASAP status, 5. Company affinity.

Parameter

- `graph` : The graph.
- `tasks` : The tasks.

Returns: The resulting collection.

Task_Weaver_Backend.Services.Algorithm.DefaultScheduleAlgorithm

Default implementation of the scheduling algorithm. Implements a deterministic backtracking algorithm with cognitive guards and multi-tenant support.

`M:Task_Weaver_Backend.Services.Algorithm.DefaultScheduleAlgorithm.GenerateScheduleAsync(System.Collections.Generic.List<Task_Weaver_Backend.Models.Task>,System.Collections.Generic.List<Task_Weaver_Backend.Models.Blocker>,System.Collections.Generic.List<Task_Weaver_Backend.Models.WorkingSchedule>)`
Generates a schedule for the given tasks, blockers, and working schedules.

Parameter

- `tasks` : The tasks.
- `blockers` : The blockers.
- `workingSchedules` : The working schedules.
- `chunkSettings` : The chunk settings.
- `isFreelancer` : The is freelancer.
- `timeZoneId` : The time zone id.
- `cancellationToken` : The cancellation token.
- `reschedulePastChunks` : The reschedule past chunks.

Returns: A task that resolves to the requested result.

`M:Task_Weaver_Backend.Services.Algorithm.DefaultScheduleAlgorithm.GetDateFromWeek(System.Int32,System.Int32,System.DayOfWeek)`

Returns the calendar date for a given ISO 8601 week number, year, and day of the week. ISO 8601: Week 1 is the week that contains the first Thursday of the year. Monday is day 1.

`M:Task_Weaver_Backend.Services.Algorithm.DefaultScheduleAlgorithm.PurgeFutureDynamicBlocks(System.Collections.Generic.List<Task_Weaver_Backend.Services.Algorithm.Models.DynamicBlock>)`
Removes all future time blocks belonging to dynamic tasks from the working block list. This enables zero-based re-planning: the scheduler places all dynamic task chunks fresh from

`M:Task_Weaver_Backend.Services.Algorithm.DefaultScheduleAlgorithm.ResolveEffectiveChunkSizes(Task_Weaver_Backend.Services.Algorithm.Models.SchedulingParameters)`
Resolves effective chunk sizes for a task with strict local-first precedence. If local values are set, they are always used. Global settings are only fallback.

Task_Weaver_Backend.Services.Algorithm.Exceptions.CascadingDeadlineClashException

Thrown when a task cannot meet its deadline due to the placement of higher-priority tasks.

`M:Task_Weaver_Backend.Services.Algorithm.Exceptions.CascadingDeadlineClashException.#ctor(System.String,System.DateTime)`

Performs cascading deadline clash exception.

Parameter

- `taskId` : The task identifier.
- `deadline` : The deadline that cannot be met.

Task_Weaver_Backend.Services.Algorithm.Exceptions.ChunkConstraintViolationException

Thrown when a task's duration cannot be divided into chunks that respect min/max constraints.

`M:Task_Weaver_Backend.Services.Algorithm.Exceptions.ChunkConstraintViolationException.#ctor(System.String,System.TimeSpan,System.TimeSpan,System.TimeSpan)`
Performs chunk constraint violation exception.

Parameter

- `taskId` : The task identifier.
- `totalDuration` : The total duration.
- `minChunk` : The min chunk.
- `maxChunk` : The maximum allowed chunk duration.

Task_Weaver_Backend.Services.Algorithm.Exceptions.CognitiveLimitExceededException

Thrown when a task cannot be scheduled because its duration or minimum chunk size exceeds the daily cognitive budget or real-time limit for its energy level.

```
M:Task_Weaver_Backend.Services.Algorithm.Exceptions.CognitiveLimitExceededException.#ctor(System.String,System.String)
```

Performs cognitive limit exceeded exception.

Parameter

- `taskId` : The task identifier.
- `reason` : The reason why cognitive limits are exceeded.

Task_Weaver_Backend.Services.Algorithm.Exceptions.CyclicDependencyException

Thrown when a circular dependency is detected in the task graph.

```
M:Task_Weaver_Backend.Services.Algorithm.Exceptions.CyclicDependencyException.#ctor(System.Collections.Generic.List{System.String})
```

Performs cyclic dependency exception.

Parameter

- `affectedTaskIds` : The affected task ids.

Task_Weaver_Backend.Services.Algorithm.Exceptions.InitialDeadlineViolationException

Thrown when a task's deadline is already violated before scheduling begins. This is a user error and will NOT be masked.

```
M:Task_Weaver_Backend.Services.Algorithm.Exceptions.InitialDeadlineViolationException.#ctor(System.String,System.DateTime)
```

Performs initial deadline violation exception.

Parameter

- `taskId` : The task identifier.
- `deadline` : The violated deadline.

Task_Weaver_Backend.Services.Algorithm.Exceptions.OrphanDependencyException

Thrown when a task references a dependency that does not exist.

```
M:Task_Weaver_Backend.Services.Algorithm.Exceptions.OrphanDependencyException.#ctor(System.String,System.String)
```

Performs orphan dependency exception.

Parameter

- `taskId` : The task identifier.
- `missingDependencyId` : The missing dependency identifier.

Task_Weaver_Backend.Services.Algorithm.Exceptions.PlanImpossibleException

Thrown when the scheduling algorithm cannot find a valid schedule. This is a complex internal error and will be masked.

```
M:Task_Weaver_Backend.Services.Algorithm.Exceptions.PlanImpossibleException.#ctor(System.String)
```

Performs plan impossible exception.

Parameter

- `reason` : The reason.

Task_Weaver_Backend.Services.Algorithm.Exceptions.PlanningException

Base exception for all planning-related errors.

```
M:Task_Weaver_Backend.Services.Algorithm.Exceptions.PlanningException.#ctor(System.String,System.String,System.Boolean,System.Collections.Generic.List{System.String})
```

Performs planning exception.

Parameter

- `errorCode` : The error code.
- `message` : The message.
- `isMasked` : The is masked.
- `affectedTaskIds` : The affected task ids.
- `slot` : The slot.
- `details` : The details.
- `userReason` : The user reason.

```
P:Task_Weaver_Backend.Services.Algorithm.Exceptions.PlanningException.AffectedTaskIds
```

List of affected task IDs (for debugging).

```
P:Task_Weaver_Backend.Services.Algorithm.Exceptions.PlanningException.IsMasked
```

Indicates whether this exception should be masked with a generic message. True = "Planning impossible" (complex internal errors). False = Original message passed through (user errors).

`P:Task_Weaver_Backend.Services.Algorithm.Exceptions.PlanningException.ReasonDetail`
Specific reason for the planning failure (for debugging).

`P:Task_Weaver_Backend.Services.Algorithm.Exceptions.PlanningException.Slot`
The slot where the error occurred (if applicable).

Task_Weaver_Backend.Services.Algorithm.Exceptions.ScheduleTimeoutException

Thrown when the scheduling algorithm times out. This is a complex internal error and will be masked.

`M:Task_Weaver_Backend.Services.Algorithm.Exceptions.ScheduleTimeoutException.#ctor`
Performs schedule timeout exception.

Task_Weaver_Backend.Services.Algorithm.Exceptions.StaticClashException

Thrown when static tasks overlap with each other or with blockers. This is a user error and will NOT be masked.

`M:Task_Weaver_Backend.Services.Algorithm.Exceptions.StaticClashException.#ctor(System.String,System.Collections.Generic.List{System.String})`
Performs static clash exception.

Parameter

- `message` : The message.
- `affectedTaskIds` : The affected task ids.

Task_Weaver_Backend.Services.Algorithm.Exceptions.TimeParadoxException

Thrown when a time paradox is detected (e.g., sum of task durations exceeds time window). This is a user error and will NOT be masked.

`M:Task_Weaver_Backend.Services.Algorithm.Exceptions.TimeParadoxException.#ctor(System.String,System.Collections.Generic.List{System.String})`
Performs time paradox exception.

Parameter

- `message` : The message.
- `affectedTaskIds` : The affected task ids.

Task_Weaver_Backend.Services.Algorithm.Exceptions.UserPlanningException

Thrown for user-actionable planning constraints that should be shown unmasked.

`M:Task_Weaver_Backend.Services.Algorithm.Exceptions.UserPlanningException.#ctor(System.String,System.Collections.Generic.List{System.String},System.IEnumerable{System.String})`
Performs user planning exception.

Parameter

- `reason` : The reason.
- `affectedTaskIds` : The affected task ids.
- `errorCode` : The error code.
- `details` : The details.

Task_Weaver_Backend.Services.Algorithm.Models.BacktrackingEntry

Entry in the backtracking stack for undo operations.

`P:Task_Weaver_Backend.Services.Algorithm.Models.BacktrackingEntry.CooldownStatusBeforePlacement`
The cooldown status before this placement (for restoration during backtracking).

`P:Task_Weaver_Backend.Services.Algorithm.Models.BacktrackingEntry.LastScheduledTaskDayBeforePlacement`
Snapshot of the day for the last scheduled task before this placement.

`P:Task_Weaver_Backend.Services.Algorithm.Models.BacktrackingEntry.LastScheduledTaskIdBeforePlacement`
Snapshot of the last scheduled task id before this placement (for restoring alternation tracker on backtrack).

`P:Task_Weaver_Backend.Services.Algorithm.Models.BacktrackingEntry.PlacedBlock`
The time block that was placed.

P:Task_Weaver_Backend.Services.Algorithm.Models.BacktrackingEntry.SearchCursorBeforePlacement
Cursor position before this placement.

P:Task_Weaver_Backend.Services.Algorithm.Models.BacktrackingEntry.Task
The task that was placed.

P:Task_Weaver_Backend.Services.Algorithm.Models.BacktrackingEntry.VisitedSlot
The slot that was visited (for visited-set tracking).

Task_Weaver_Backend.Services.Algorithm.Models.ChunkSettings

Global chunk size configuration for flow protection.

P:Task_Weaver_Backend.Services.Algorithm.Models.ChunkSettings.GlobalMaxChunkSize
Global maximum chunk size.

P:Task_Weaver_Backend.Services.Algorithm.Models.ChunkSettings.GlobalMinChunkSize
Global minimum chunk size.

Task_Weaver_Backend.Services.Algorithm.Models.CognitiveLevel

Represents the cognitive difficulty level of a task. Used for cognitive budget calculations in the scheduling algorithm.

F:Task_Weaver_Backend.Services.Algorithm.Models.CognitiveLevel.LEVEL_1_NORMAL
Normal or light tasks that require minimal cognitive effort. No cognitive budget cost.

F:Task_Weaver_Backend.Services.Algorithm.Models.CognitiveLevel.LEVEL_2_HEAVY
Heavy tasks that require significant cognitive effort. Budget cost: Duration × 1, Real-time limit: 240 minutes/day.

F:Task_Weaver_Backend.Services.Algorithm.Models.CognitiveLevel.LEVEL_3_VERY_HEAVY
Very heavy tasks that require intense cognitive effort. Budget cost: Duration × 2, Real-time limit: 120 minutes/day.

Task_Weaver_Backend.Services.Algorithm.Models.CpmResult

Result of Critical Path Method (CPM) calculation for a task.

P:Task_Weaver_Backend.Services.Algorithm.Models.CpmResult.EarliestStart
Earliest start time for the task.

P:Task_Weaver_Backend.Services.Algorithm.Models.CpmResult.LatestFinish
Latest finish time for the task.

P:Task_Weaver_Backend.Services.Algorithm.Models.CpmResult.Slack
Slack time (buffer) available for the task. Negative slack indicates a deadline violation.

P:Task_Weaver_Backend.Services.Algorithm.Models.CpmResult.TaskId
Task identifier.

Task_Weaver_Backend.Services.Algorithm.Models.DependencyGraph

Represents a directed acyclic graph (DAG) of task dependencies.

M:Task_Weaver_Backend.Services.Algorithm.Models.DependencyGraph.AreDependenciesMet(System.String,System.Collections.Generic.HashSet{System.String})
Checks if all dependencies for a given task are met.

Parameter

- taskId : The task identifier.
- completedTaskIds : The completed task ids.

Returns: True if all dependencies are met, false otherwise.

P:Task_Weaver_Backend.Services.Algorithm.Models.DependencyGraph.Edges
Dictionary of edges (TaskId -> List of dependent TaskIds).

`P:Task_Weaver_Backend.Services.Algorithm.Models.DependencyGraph.HasCycle`
Indicates whether the graph contains a cycle.

`P:Task_Weaver_Backend.Services.Algorithm.Models.DependencyGraph.Nodes`
Dictionary of task nodes (TaskId -> Task).

Task_Weaver_Backend.Services.Algorithm.Models.IntervalTree

Interval tree for efficient blocked time range queries. Stores merged, non-overlapping time intervals.

`M:Task_Weaver_Backend.Services.Algorithm.Models.IntervalTree.Add(System.DateTime,System.DateTime)`
Adds a time interval to the tree.

Parameter

- `start` : The start.
- `end` : The end.

`M:Task_Weaver_Backend.Services.Algorithm.Models.IntervalTree.GetIntervals`
Gets all intervals in the tree.

Returns: A read-only list of all stored intervals.

`M:Task_Weaver_Backend.Services.Algorithm.Models.IntervalTree.IsBlocked(System.DateTime,System.DateTime)`
Checks if a given time range is blocked by any interval in the tree.

Parameter

- `start` : The start.
- `end` : The end.

Returns: True when the condition is met; otherwise false.

`M:Task_Weaver_Backend.Services.Algorithm.Models.IntervalTree.Merge`
Merges overlapping and adjacent intervals into a single sorted list. Must be called after all intervals are added.

Task_Weaver_Backend.Services.Algorithm.Models.OrganizationRules

Organization-specific rules for task scheduling (e.g., freelancer constraints).

`P:Task_Weaver_Backend.Services.Algorithm.Models.OrganizationRules.AllowedOrganizationIds`
List of organization IDs the user is allowed to work for.

`P:Task_Weaver_Backend.Services.Algorithm.Models.OrganizationRules.CanUseFlexSlots`
Indicates whether the user can use flex slots (non-exclusive time blocks).

`P:Task_Weaver_Backend.Services.Algorithm.Models.OrganizationRules.IsFreelancer`
Indicates whether the user is a freelancer. Derived explicitly from `UserProfileDto.SystemRole == "freelancer"`.

Task_Weaver_Backend.Services.Algorithm.Models.ScheduleProfile

Bundled user context and configuration for the scheduling algorithm.

`P:Task_Weaver_Backend.Services.Algorithm.Models.ScheduleProfile.ChunkSettings`
Global chunk size settings.

`P:Task_Weaver_Backend.Services.Algorithm.Models.ScheduleProfile.OrganizationRules`
Organization-specific rules (freelancer constraints, etc.).

`P:Task_Weaver_Backend.Services.Algorithm.Models.ScheduleProfile.SlotGranularity`
Slot granularity for rounding (e.g., 5, 10, or 15 minutes).

`P:Task_Weaver_Backend.Services.Algorithm.Models.ScheduleProfile.WorkPlan`
User's working schedule configuration.

Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask

Internal task model used by the scheduling algorithm. Isolated from the Firestore DTO layer.

`F:Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask.AsapPriorityLevel`
Priority level that marks a task as ASAP (As Soon As Possible).

`P:Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask.CognitiveLevel`
Cognitive difficulty level of the task.

`P:Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask.Dependencies`
List of task IDs that must be completed before this task can start.

`P:Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask.HardDeadline`
Hard deadline for task completion.

`P:Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask.IsAsap`
Indicates whether this task has ASAP priority (Priority == 10).

`P:Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask.IsCompleted`
Indicates whether the task is fully completed.

`P:Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask.IsPastLocked`
Indicates whether past blocks of this task are locked and cannot be rescheduled.

`P:Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask.IsStatic`
Indicates whether this task is static (manually fixed) and cannot be moved.

`P:Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask.LocalMaxChunkSize`
Task-specific maximum chunk size override (null = use global setting).

`P:Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask.LocalMinChunkSize`
Task-specific minimum chunk size override (null = use global setting).

`P:Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask.OrganizationId`
Organization identifier this task belongs to.

`P:Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask.PreferImmediateContinuation`
Optional scheduling hint to prioritize immediate continuation of this task.

`P:Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask.Priority`
Task priority (1-10, where 10 is ASAP).

`P:Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask.RemainingDuration`
Remaining duration to be scheduled (decreases as chunks are placed).

`P:Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask.TaskId`
Unique identifier of the task.

`P:Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask.Title`
Human-readable title of the task.

`P:Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask.TotalDuration`
Total duration of the task in minutes.

Task_Weaver_Backend.Services.Algorithm.Models.TimeBlock

Represents a time block (blocker or scheduled task chunk).

`P:Task_Weaver_Backend.Services.Algorithm.Models.TimeBlock.EndTime`
End time of the block (UTC).

`P:Task_Weaver_Backend.Services.Algorithm.Models.TimeBlock.IsExclusiveSlot`
Indicates whether this slot is exclusive to a specific organization (for freelancers).

`P:Task_Weaver_Backend.Services.Algorithm.Models.TimeBlock.IsManuallyMoved`
True when the block was manually placed by drag-and-drop and should remain fixed during dynamic re-scheduling.

`P:Task_Weaver_Backend.Services.Algorithm.Models.TimeBlock.IsPastLocked`
Indicates whether this block is in the past and locked.

`P:Task_Weaver_Backend.Services.Algorithm.Models.TimeBlock.IsTrackedExecution`
True when the block comes from concrete timestamped execution evidence (ISO datetime), not from a planned HH:mm scheduler slot.

`P:Task_Weaver_Backend.Services.Algorithm.Models.TimeBlock.OrganizationId`
Organization ID this block is associated with.

`P:Task_Weaver_Backend.Services.Algorithm.Models.TimeBlock.StartTime`
Start time of the block (UTC).

`P:Task_Weaver_Backend.Services.Algorithm.Models.TimeBlock.TaskId`
Task ID this block belongs to (empty for blockers).

Task_Weaver_Backend.Services.Algorithm.Models.WorkPlan

Represents the user's working schedule configuration.

`P:Task_Weaver_Backend.Services.Algorithm.Models.WorkPlan.DailySlots`
Organization-specific time slots (for freelancers with multiple clients). Key: DateOnly, Value: List of time slots with optional organization filter.

`P:Task_Weaver_Backend.Services.Algorithm.Models.WorkPlan.WorkDayEnd`
End time of the work day.

`P:Task_Weaver_Backend.Services.Algorithm.Models.WorkPlan.WorkDayStart`
Start time of the work day.

`P:Task_Weaver_Backend.Services.Algorithm.Models.WorkPlan.WorkDays`
Days of the week the user works.

Task_Weaver_Backend.Services.Algorithm.PreProcessor

Preprocessor for normalizing input data and applying delta scheduling.

`M:Task_Weaver_Backend.Services.Algorithm.PreProcessor.ApplyDeltaScheduling(System.Collections.Generic.List{Task_Weaver_Backend.Services.Algorithm.Models.TaskBlock})`
Applies delta scheduling: locks past blocks and subtracts completed time from total duration. Future blocks of dynamic tasks are NOT subtracted here — they will be purged from the working block list before re-planning, ensuring a clean zero-based re-schedule from

Parameter

- `tasks` : The tasks.
- `existingBlocks` : The existing blocks.
- `now` : The now.
- `reschedulePastChunks` : When

`M:Task_Weaver_Backend.Services.Algorithm.PreProcessor.NormalizeToUtc(System.Collections.Generic.List{Task_Weaver_Backend.Services.Algorithm.Models.TaskBlock})`
Normalizes all datetime values to UTC.

Parameter

- `tasks` : The tasks.
- `blocks` : The blocks.

`M:Task_Weaver_Backend.Services.Algorithm.PreProcessor.UnrollAndMergeSperrzonen(System.Collections.Generic.List{Task_Weaver_Backend.Services.Algorithm.Models.TaskBlock})`
Unrolls recurring blockers and merges all blocked time ranges into an interval tree.

Parameter

- `blockers` : The blockers.
- `staticTasks` : The static tasks.
- `now` : The now.

Returns: The computed result.

Task_Weaver_Backend.Services.Algorithm.SlotFinder

Finds eligible time slots for task placement with organization-specific constraints.

`M:Task_Weaver_Backend.Services.Algorithm.SlotFinder.#ctor(Task_Weaver_Backend.Services.Algorithm.CalendarGrid,Task_Weaver_Backend.Services.Algorithm.Models.TaskBlock)`
Performs slot finder.

Parameter

- `calendarGrid` : The calendar grid.

- `orgRules` : The org rules.
- `workPlan` : The work plan.

`M:Task_Weaver_Backend.Services.Algorithm.SlotFinder.ApplyGranularityRounding(System.DateTime, System.TimeSpan)`
Rounds a slot to the specified granularity (5, 10, or 15 minutes).

Parameter

- `slot` : The slot.
- `granularity` : The granularity.

Returns: The computed result.

`M:Task_Weaver_Backend.Services.Algorithm.SlotFinder.FindNextEligibleSlot(Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask, System.DateTime)`
Finds the next eligible slot for a task, considering all constraints.

Parameter

- `task` : The task.
- `from` : The from.
- `visitedSet` : The visited set.
- `slotGranularity` : The slot granularity.

Returns: The computed result.

`M:Task_Weaver_Backend.Services.Algorithm.SlotFinder.IsBeforeDeadline(System.DateTime, System.DateTime)`
Checks if a slot is before the deadline.

`M:Task_Weaver_Backend.Services.Algorithm.SlotFinder.IsInVisitedSet(System.String, System.DateTime, System.Collections.Generic.HashSet<System.String>)`
Checks if a task-slot combination is in the visited set.

`M:Task_Weaver_Backend.Services.Algorithm.SlotFinder.IsSlotExclusiveForOtherOrganization(System.DateTime, Task_Weaver_Backend.Services.Algorithm.Models.SchedulingTask)`
Checks if a slot is exclusive to a different organization (for freelancers).

`M:Task_Weaver_Backend.Services.Algorithm.SlotFinder.IsWithinWorkPlan(System.DateTime)`
Checks if a slot is within the defined work plan hours.

Task_Weaver_Backend.Services.BlockerService

Provides CRUD operations for blockers and keeps schedules in sync.

`M:Task_Weaver_Backend.Services.BlockerService.#ctor(Task_Weaver_Backend.Models.IBlockerRepository, Task_Weaver_Backend.Models.ITaskRepository, Task_Weaver_Backend.Models.IUserRepository, Task_Weaver_Backend.Models.IValidator, Task_Weaver_Backend.Models.IScheduler)`
Initializes a new instance of the

Parameter

- `blockerRepository` : Repository used to persist blocker entities.
- `taskRepository` : Repository used for overlap checks with tasks.
- `userRepository` : Repository used to load user context data.
- `validator` : Validator used to enforce blocker input rules.
- `scheduler` : Scheduler used to recompute plans after blocker changes.

`M:Task_Weaver_Backend.Services.BlockerService.CreateCalendarBlockerAsync(System.String, Task_Weaver_Backend.Models.BlockerDto)`
Creates a blocker, validates overlaps, and re-schedules tasks.

Parameter

- `uid` : The user identifier.
- `createDto` : Payload containing blocker properties.

Returns: A task that represents the asynchronous operation.

`M:Task_Weaver_Backend.Services.BlockerService.DeleteCalendarBlockerAsync(System.String, System.String)`
Deletes a blocker and triggers task re-scheduling.

Parameter

- `uid` : The user identifier.
- `blockerId` : The blocker identifier.

Returns: A task that represents the asynchronous operation.

`M:Task_Weaver_Backend.Services.BlockerService.GetCalendarBlockersAsync(System.String)`
Loads all blockers for the given user.

Parameter

- `uid` : The user identifier.

Returns: List of blockers assigned to the user calendar.

`M:Task_Weaver_Backend.Services.BlockerService.PatchCalendarBlockerAsync(System.String, System.String, Task_Weaver_Backend.Models.BlockerDto)`
Updates an existing blocker, validates overlaps, and re-schedules tasks.

Parameter

- `uid` : The user identifier.
- `blockerId` : The blocker identifier.
- `updateDto` : Payload containing updated blocker properties.

Returns: A task that represents the asynchronous operation.

Task_Weaver_Backend.Services.BrevoEmailService

Represents the brevo email service.

M:Task_Weaver_Backend.Services.BrevoEmailService.#ctor(System.Net.Http.HttpClient,Task_Weaver_Backend.Services.IEnvVariableLoader)
Performs brevo email service.

Parameter

- `http` : The http.
- `envVariableLoader` : The env variable loader.

M:Task_Weaver_Backend.Services.BrevoEmailService.SendInviteEmailAsync(System.String,System.String,System.String)
Performs send invite email async.

Parameter

- `toEmail` : The recipient email address.
- `inviteId` : The invite identifier.
- `organizationName` : The organization name.

Returns: A task that represents the asynchronous operation.

Task_Weaver_Backend.Services.IBlockerService

interface IBlockerService.

M:Task_Weaver_Backend.Services.IBlockerService.CreateCalendarBlockerAsync(System.String,Task_Weaver_Backend.Models.BlockerDto)
Erstellt einen Kalender-Blocker fuer den Nutzer.

Parameter

- `uid` : Firebase-UID des Nutzers.
- `createDto` : Blocker-Daten.

M:Task_Weaver_Backend.Services.IBlockerService.DeleteCalendarBlockerAsync(System.String,System.String)
Loescht einen Kalender-Blocker.

Parameter

- `uid` : Firebase-UID des Nutzers.
- `blockerId` : Kennung des Blockers.

M:Task_Weaver_Backend.Services.IBlockerService.GetCalendarBlockersAsync(System.String)
Laedt alle Kalender-Blocker des Nutzers.

Parameter

- `uid` : Firebase-UID des Nutzers.

Returns: Liste aller Blocker.

M:Task_Weaver_Backend.Services.IBlockerService.PatchCalendarBlockerAsync(System.String,System.String,Task_Weaver_Backend.Models.BlockerDto)
Aktualisiert einen vorhandenen Kalender-Blocker.

Parameter

- `uid` : Firebase-UID des Nutzers.
- `blockerId` : Kennung des Blockers.
- `updateDto` : Neue Blocker-Daten.

Task_Weaver_Backend.Services.IEmailService

interface IEmailService.

M:Task_Weaver_Backend.Services.IEmailService.SendInviteEmailAsync(System.String,System.String,System.String)
Send a transactional email.

Task_Weaver_Backend.Services.IEnvVariableLoader

interface IEnvVariableLoader.

M:Task_Weaver_Backend.Services.IEnvVariableLoader.Load(System.String)
Performs load.

Parameter

- `variableName` : The variable name.

Returns: The requested load.

Task_Weaver_Backend.Services.IOrganizationService

interface IOrganizationService.

M:Task_Weaver_Backend.Services.IOrganizationService.CreateInviteAsync(System.String,System.String,System.String)

Creates an invitation to join the organization. Requires admin role.

Parameter

- **uid** : Requesting user identifier.
- **orgId** : Organization identifier.
- **email** : Email address of the invitee.

Returns: The generated invitation token.

Exceptions

- **T:Task_Weaver_Backend.Models.AppException** : Thrown when user is not an admin or organization not found.

M:Task_Weaver_Backend.Services.IOrganizationService.DeleteInviteAsync(System.String,System.String,System.String)

Deletes a pending invitation. Requires admin role.

Parameter

- **uid** : Requesting user identifier.
- **orgId** : Organization identifier.
- **inviteId** : Invitation identifier.

Exceptions

- **T:Task_Weaver_Backend.Models.AppException** : Thrown when user is not an admin or invitation not found.

M:Task_Weaver_Backend.Services.IOrganizationService.GetOrganizationAsync(System.String,System.String)

Retrieves organization details. Requires membership.

Parameter

- **uid** : Requesting user identifier.
- **orgId** : Organization identifier.

Returns: Organization details.

Exceptions

- **T:Task_Weaver_Backend.Models.AppException** : Thrown when user is not a member or organization not found.

M:Task_Weaver_Backend.Services.IOrganizationService.GetOrganizationMembersAsync(System.String,System.String)

Retrieves all members of an organization. Requires admin role.

Parameter

- **uid** : Requesting user identifier.
- **orgId** : Organization identifier.

Returns: List of organization members with role information.

Exceptions

- **T:Task_Weaver_Backend.Models.AppException** : Thrown when user is not an admin or organization not found.

M:Task_Weaver_Backend.Services.IOrganizationService.GetOrganizationSettingsAsync(System.String,System.String)

Retrieves organization settings. Requires admin role.

Parameter

- **uid** : Requesting user identifier.
- **orgId** : Organization identifier.

Returns: Organization settings.

Exceptions

- **T:Task_Weaver_Backend.Models.AppException** : Thrown when user is not an admin or organization not found.

M:Task_Weaver_Backend.Services.IOrganizationService.GetPendingInvitesAsync(System.String,System.String)

Retrieves all pending invitations for an organization. Requires admin role.

Parameter

- **uid** : Requesting user identifier.
- **orgId** : Organization identifier.

Returns: List of pending invitations.

Exceptions

- **T:Task_Weaver_Backend.Models.AppException** : Thrown when user is not an admin or organization not found.

M:Task_Weaver_Backend.Services.IOrganizationService.RemoveMemberAsync(System.String,System.String,System.String)

Removes a member from the organization. Requires admin role. For employees (systemRole=employee), deletes from both Firestore and Firebase Auth. For freelancers (systemRole=freelancer), only removes from organization.

Parameter

- **uid** : Requesting user identifier.
- **orgId** : Organization identifier.
- **targetUid** : User identifier to remove.

Exceptions

- **T:Task_Weaver_Backend.Models.AppException** : Thrown when user is not an admin, target is last admin, or member not found.

M:Task_Weaver_Backend.Services.IOrganizationService.UpdateMemberRoleAsync(System.String,System.String,System.String,System.Boolean)

Updates the role of an existing member. Requires admin role.

Parameter

- **uid** : Requesting user identifier.
- **orgId** : Organization identifier.
- **targetUid** : User identifier to update.
- **isAdmin** : Whether the target user should have admin privileges.

Exceptions

- `T:Task_Weaver_Backend.Models.AppException` : Thrown when user is not an admin, target is last admin being demoted, or member not found.

`M:Task_Weaver_Backend.Services.IOrganizationService.UpdateOrganizationSettingsAsync(System.String,System.String,Task_Weaver_Backend.Models.Organizat`

Updates organization settings. Requires admin role.

Parameter

- `uid` : Requesting user identifier.
- `orgId` : Organization identifier.
- `settings` : Updated settings payload.

Returns:

Updated organization settings.

Exceptions

- `T:Task_Weaver_Backend.Models.AppException` : Thrown when user is not an admin, whitelist conflict occurs, or organization not found.

Task_Weaver_Backend.Services.IScheduleAlgorithm

interface IScheduleAlgorithm.

`M:Task_Weaver_Backend.Services.IScheduleAlgorithm.GenerateScheduleAsync(System.Collections.Generic.List{Task_Weaver_Backend.Models.DTOs.TaskDto},Sys`

Generates a schedule for the given tasks using blockers, availability, and chunk settings.

Parameter

- `tasks` : Tasks that should be scheduled.
- `blockers` : Blocking intervals that must not be used.
- `workingSchedules` : User working schedules that define available windows.
- `chunkSettings` : Global chunk boundaries for splitting work.
- `isFreelancer` : Whether freelancer scheduling rules should be applied.
- `timeZoneId` : IANA time zone identifier used for calculations.
- `cancellationToken` : Cancellation token for cooperative cancellation.
- `reschedulePastChunks` : When

Returns:

Scheduled tasks including their computed date chunks.

Task_Weaver_Backend.Services.IScheduler

Service responsible for orchestrating the rescheduling of tasks.

`M:Task_Weaver_Backend.Services.IScheduler.RescheduleTasksAsync(System.String,System.Collections.Generic.List{Task_Weaver_Backend.Models.DTOs.TaskDto}`

Reschedules tasks for a specific user, considering blockers and working schedules.

Parameter

- `uid` : The user identifier.
- `tasks` : The tasks.
- `blockers` : The blockers.
- `workingSchedules` : The working schedules.
- `chunkSettingsOverride` : The chunk settings override.
- `reschedulePastChunks` : When
- `cancellationToken` : The cancellation token.

Returns:

A task representing the asynchronous operation.

Task_Weaver_Backend.Services.ITaskService

interface ITaskService.

`M:Task_Weaver_Backend.Services.ITaskService.CreateCalendarTaskAsync(System.String,Task_Weaver_Backend.Models.DTOs.CreateTaskDto)`

Creates a new calendar task.

Parameter

- `uid` : Firebase UID of the user.
- `taskData` : Task to be created.

Returns:

The created task.

`M:Task_Weaver_Backend.Services.ITaskService.DeleteCalendarTaskAsync(System.String,System.String)`

Deletes a calendar task.

Parameter

- `uid` : Firebase UID of the user.
- `taskId` : Identifier of the task.

`M:Task_Weaver_Backend.Services.ITaskService.GetCalendarTasksAsync(System.String)`

Loads all calendar tasks of the user.

Parameter

- `uid` : Firebase UID of the user.

Returns:

List of all tasks.

`M:Task_Weaver_Backend.Services.ITaskService.MoveTaskChunkAsync(System.String,System.String,Task_Weaver_Backend.Models.DTOs.MoveTaskChunkDto)`

Moves a scheduled task chunk to a new date/time range.

Parameter

- `uid` : Firebase UID of the user.
- `taskId` : Identifier of the task.
- `newRange` : New date/time range.

- `uid` : Firebase UID of the user.
- `taskId` : Identifier of the task.
- `dto` : Source and target chunk coordinates.

`M:Task_Weaver_Backend.Services.ITaskService.PatchCalendarTaskAsync(System.String,System.String,Task_Weaver_Backend.Models.DTos.TaskDto,System.Boolean)`
Selectively updates individual fields of an existing calendar task.

Parameter

- `uid` : Firebase UID of the user.
- `taskId` : Identifier of the task.
- `taskData` : Only the task fields sent in the request.
- `reschedulePastChunks` : When

`M:Task_Weaver_Backend.Services.ITaskService.PatchCalendarTaskDatesAsync(System.String,System.String,System.Collections.Generic.List{Task_Weaver_Backend.Models.DTos.TaskDto})`
Replaces the Dates array of an existing calendar task.

Task_Weaver_Backend.Services.IUserService

Encapsulates the business logic for user profile, organization assignment, calendar data, and work profiles.

`M:Task_Weaver_Backend.Services.IUserService.AcceptInviteAsync(System.String,System.String)`
Accepts an organization invitation via its invite ID.

Parameter

- `uid` : Firebase UID of the user.
- `inviteId` : Invite ID of the organization.

`M:Task_Weaver_Backend.Services.IUserService.GetMyOrganizationsAsync(System.String)`
Loads all organizations of the authenticated user including role and join date.

Parameter

- `uid` : Firebase UID of the user.

Returns: List of all assigned organizations.

`M:Task_Weaver_Backend.Services.IUserService.GetUserAsync(System.String)`
Loads the profile of the authenticated user.

Parameter

- `uid` : Firebase UID of the user.

Returns: The user profile.

`M:Task_Weaver_Backend.Services.IUserService.LeaveOrganizationAsync(System.String,System.String)`
Voluntarily removes the user from an organization.

Parameter

- `uid` : Firebase UID of the user.
- `orgId` : ID of the organization.

Exceptions

- `T:Task_Weaver_Backend.Models.AppException` : Thrown if the organization does not exist, the user is not a member, or the user tries to leave as the last remaining admin.

`M:Task_Weaver_Backend.Services.IUserService.RegisterUserAsync(System.String,Task_Weaver_Backend.Models.DTos.RegisterUserDto)`
Creates or synchronizes the Firestore user profile for the authenticated user.

Parameter

- `uid` : Firebase UID of the user.
- `registrationData` : User profile to be created.

`M:Task_Weaver_Backend.Services.IUserService.UpdateUserAsync(System.String,Task_Weaver_Backend.Models.UserDto)`
Updates the personal profile data of the user.

Parameter

- `uid` : Firebase UID of the user.
- `updateData` : Profile changes to be saved.

Task_Weaver_Backend.Services.IWorkProfileService

interface IWorkProfileService.

`M:Task_Weaver_Backend.Services.IWorkProfileService.CreateWorkingScheduleAsync(System.String,Task_Weaver_Backend.Models.WorkingScheduleDto)`
Creates a new working schedule including overlap check.

Parameter

- `uid` : Firebase UID of the user.
- `schedule` : Working schedule to be created.

Returns: The created working schedule.

`M:Task_Weaver_Backend.Services.IWorkProfileService.DeleteWorkingScheduleAsync(System.String,System.String)`
Deletes a working schedule.

Parameter

- `uid` : Firebase UID of the user.
- `scheduleId` : Identifier of the working schedule.

`M:Task_Weaver_Backend.Services.IWorkProfileService.GetChunkSettingsAsync(System.String)`
Loads the user's global chunk settings.

Parameter

- `uid` : Firebase UID of the user.

Returns: The global chunk settings or

`M:Task_Weaver_Backend.Services.IWorkProfileService.GetWorkingSchedulesAsync(System.String)`
Loads all working schedules of the user.

Parameter

- `uid` : Firebase UID of the user.

Returns: List of all working schedules.

`M:Task_Weaver_Backend.Services.IWorkProfileService.PatchChunkSettingsAsync(System.String,Task_Weaver_Backend.Models.GlobalChunkSettingsDto)`
Updates the global chunk settings for protecting continuous work phases.

Parameter

- `uid` : Firebase UID of the user.
- `settings` : New chunk settings.

Returns: The updated chunk settings.

`M:Task_Weaver_Backend.Services.IWorkProfileService.PatchWorkingScheduleAsync(System.String,System.String,Task_Weaver_Backend.Models.WorkingScheduleD`
Updates an existing working schedule including overlap check.

Parameter

- `uid` : Firebase UID of the user.
- `scheduleId` : Identifier of the working schedule.
- `schedule` : Updated schedule data.

Returns: The updated working schedule.

Task_Weaver_Backend.Services.OrganizationService

Represents the organization service.

`M:Task_Weaver_Backend.Services.OrganizationService.#ctor(Task_Weaver_Backend.Models.IOrganizationRepository,Task_Weaver_Backend.Models.IUserReposito`
Performs organization service.

Parameter

- `organizationRepository` : The organization repository.
- `userRepository` : The user repository.
- `organizationValidator` : The organization validator.
- `emailService` : The email service.

`M:Task_Weaver_Backend.Services.OrganizationService.CreateInviteAsync(System.String,System.String,System.String)`
Performs create invite async.

Parameter

- `uid` : The user identifier.
- `orgId` : The organization identifier.
- `email` : The email address.

Returns: A task that represents the asynchronous operation.

`M:Task_Weaver_Backend.Services.OrganizationService.DeleteInviteAsync(System.String,System.String,System.String)`
Performs delete invite async.

Parameter

- `uid` : The user identifier.
- `orgId` : The organization identifier.
- `inviteId` : The invite identifier.

Returns: A task that represents the asynchronous operation.

`M:Task_Weaver_Backend.Services.OrganizationService.GetOrganizationAsync(System.String,System.String)`
Performs get organization async.

Parameter

- `uid` : The user identifier.
- `orgId` : The organization identifier.

Returns: The requested organization async.

`M:Task_Weaver_Backend.Services.OrganizationService.GetOrganizationMembersAsync(System.String,System.String)`
Performs get organization members async.

Parameter

- `uid` : The user identifier.
- `orgId` : The organization identifier.

Returns: The requested organization members async.

`M:Task_Weaver_Backend.Services.OrganizationService.GetOrganizationSettingsAsync(System.String,System.String)`
Performs get organization settings async.

Parameter

- `uid` : The user identifier.
- `orgId` : The organization identifier.

Returns: The requested organization settings async.

`M:Task_Weaver_Backend.Services.OrganizationService.GetPendingInvitesAsync(System.String,System.String)`
Performs get pending invites async.

Parameter

- `uid` : The user identifier.
- `orgId` : The organization identifier.

Returns: The requested pending invites async.

`M:Task_Weaver_Backend.Services.OrganizationService.IsEmailInWhitelistAsync(System.String)`
Performs is email in whitelist async.

Parameter

- `email` : The email address.

Returns: The requested is email in whitelist async.

`M:Task_Weaver_Backend.Services.OrganizationService.RemoveMemberAsync(System.String,System.String,System.String)`
Performs remove member async.

Parameter

- `uid` : The user identifier.
- `orgId` : The organization identifier.
- `targetUid` : The user identifier.

Returns: A task that represents the asynchronous operation.

`M:Task_Weaver_Backend.Services.OrganizationService.UpdateMemberRoleAsync(System.String,System.String,System.String,System.Boolean)`
Performs update member role async.

Parameter

- `uid` : The user identifier.
- `orgId` : The organization identifier.
- `targetUid` : The user identifier.
- `isAdmin` : The is admin.

Returns: A task that represents the asynchronous operation.

`M:Task_Weaver_Backend.Services.OrganizationService.UpdateOrganizationSettingsAsync(System.String,System.String,Task_Weaver_Backend.Models.OrganizationSettings)`
Performs update organization settings async.

Parameter

- `uid` : The user identifier.
- `orgId` : The organization identifier.
- `settings` : The settings.

Returns: A task that represents the asynchronous operation.

Task_Weaver_Backend.Services.Scheduler

Implementation of the scheduler service.

`M:Task_Weaver_Backend.Services.Scheduler.#ctor(Task_Weaver_Backend.Models.ITaskRepository,Task_Weaver_Backend.Models.IBlockerRepository,Task_Weaver_Backend.Models.IWorkProfileRepository,Task_Weaver_Backend.Models.IUserRepository,Task_Weaver_Backend.Models.IScheduleAlgorithm)`
Initializes a new instance of the

Parameter

- `taskRepository` : The task repository.
- `blockerRepository` : The blocker repository.
- `workProfileRepository` : The work profile repository.
- `userRepository` : The user repository.
- `scheduleAlgorithm` : The schedule algorithm.

`M:Task_Weaver_Backend.Services.Scheduler.RescheduleTasksAsync(System.String,System.Collections.Generic.List{Task_Weaver_Backend.Models.DT0s.TaskDto})`
Executes reschedule tasks async.

Parameter

- `uid` : The user identifier.
- `tasks` : The tasks.
- `blockers` : The blockers.
- `workingSchedules` : The working schedules.
- `chunkSettingsOverride` : The chunk settings override.
- `reschedulePastChunks` : The reschedule past chunks.
- `cancellationToken` : The cancellation token.

Returns: A task that represents the asynchronous operation.

Task_Weaver_Backend.Services.SecretLoader

Represents the secret loader.

```
M:Task_Weaver_Backend.Services.SecretLoader.Load(System.String)
```

Performs load.

Parameter

- `secretId` : The secret id.

Returns: The requested load.

Task_Weaver_Backend.Services.ShearedService

Represents the sheared service.

```
M:Task_Weaver_Backend.Services.ShearedService.#ctor(Task_Weaver_Backend.Models.IOrganizationRepository,Task_Weaver_Backend.Models.IUserRepository)
```

Performs sheared service.

Parameter

- `organizationRepository` : The organization repository.
- `userRepository` : The user repository.

Task_Weaver_Backend.Services.TaskService

Creates a calendar task after validating the company assignment for freelancer accounts.

```
M:Task_Weaver_Backend.Services.TaskService.#ctor(Task_Weaver_Backend.Models.ITaskRepository,Task_Weaver_Backend.Models.IUserRepository,Task_Weaver_Backend.Models.IScheduler,Task_Weaver_Backend.Models.IValidator,Task_Weaver_Backend.Models.IWorkProfileRepository,Task_Weaver_Backend.Models.IBlockerRepository)
```

Performs task service.

Parameter

- `taskRepository` : The task repository.
- `userRepository` : The user repository.
- `organizationRepository` : The organization repository.
- `scheduler` : The scheduler.
- `validator` : The validator.
- `workProfileRepository` : The work profile repository.
- `blockerRepository` : The blocker repository.

```
M:Task_Weaver_Backend.Services.TaskService.CreateCalendarTaskAsync(System.String,Task_Weaver_Backend.Models.DT0s.CreateTaskDto)
```

Creates a calendar task. For static tasks, validates that the new time block does not overlap with any existing static task or blocker (within 365 days). Dynamic tasks are validated by attempting a full schedule before any persistence.

Parameter

- `uid` : The user identifier.
- `taskData` : The task data.

Returns: A task that represents the asynchronous operation.

```
M:Task_Weaver_Backend.Services.TaskService.DeleteCalendarTaskAsync(System.String,System.String)
```

Deletes an existing calendar task after verifying that it exists. No conflict validation is performed – deletion can only resolve, never create, conflicts.

Parameter

- `uid` : The user identifier.
- `taskId` : The task identifier.

Returns: A task that represents the asynchronous operation.

```
M:Task_Weaver_Backend.Services.TaskService.GetCalendarTasksAsync(System.String)
```

Returns the calendar tasks that belong to the authenticated user.

Parameter

- `uid` : The user identifier.

Returns: A task that resolves to the requested result.

```
M:Task_Weaver_Backend.Services.TaskService.MoveTaskChunkAsync(System.String,System.String,Task_Weaver_Backend.Models.DT0s.MoveTaskChunkDto)
```

Moves a single existing scheduled chunk by exact source coordinates.

Parameter

- `uid` : The user identifier.
- `taskId` : The task identifier.
- `dto` : The dto.

Returns: A task that represents the asynchronous operation.

```
M:Task_Weaver_Backend.Services.TaskService.PatchCalendarTaskAsync(System.String,System.String,Task_Weaver_Backend.Models.DT0s.TaskDto,System.Boolean)
```

Updates only the task fields that were explicitly sent by the client. For static tasks, validates that the updated time block does not create new overlaps.

Parameter

- `uid` : The user identifier.
- `taskId` : The task identifier.
- `taskData` : The task data.
- `reschedulePastChunks` : The reschedule past chunks.

Returns: A task that represents the asynchronous operation.

```
M:Task_Weaver_Backend.Services.TaskService.PatchCalendarTaskDatesAsync(System.String,System.String,System.Collections.Generic.List{Task_Weaver_Backend.Models.DT0s.TaskDto})
```

Performs patch calendar task dates async.

Parameter

- `uid` : The user identifier.
- `taskId` : The task identifier.
- `dates` : The dates.

Returns: A task that represents the asynchronous operation.

`M:Task_Weaver_Backend.Services.TaskService.ValidateNewStaticBlocksAsync(System.String,Task_Weaver_Backend.Models.DTos.TaskDto,System.String,System.T`
Validates that none of the new task's time blocks violate working schedules and that non-past blocks do not overlap with existing static tasks/blockers within the next 365 days. when true, past blocks are validated against working schedules; when false, past blocks are skipped.

Task_Weaver_Backend.Services.UserService

Business logic for user profile, organization membership, blockers and work-profile operations.

`M:Task_Weaver_Backend.Services.UserService.#ctor(Task_Weaver_Backend.Models.IUserRepository,Task_Weaver_Backend.Models.IOrganizationRepository,Task_I`
Initializes a new instance of the

Parameter

- `userRepository` : The user repository.
- `organizationRepository` : The organization repository.
- `userValidator` : The user validator.

`M:Task_Weaver_Backend.Services.UserService.AcceptInviteAsync(System.String,System.String)`
Assigns a user to an organization with the specified role.

Parameter

- `uid` : The user identifier.
- `inviteId` : The invite identifier.

Returns: A task that represents the asynchronous operation.

`M:Task_Weaver_Backend.Services.UserService.GetMyOrganizationsAsync(System.String)`
Executes get my organizations async.

Parameter

- `uid` : The user identifier.

Returns: A task that resolves to the requested result.

`M:Task_Weaver_Backend.Services.UserService.GetUserAsync(System.String)`
Performs get user async.

Parameter

- `uid` : The user identifier.

Returns: The requested user async.

`M:Task_Weaver_Backend.Services.UserService.LeaveOrganizationAsync(System.String,System.String)`
Executes leave organization async.

Parameter

- `uid` : The user identifier.
- `orgId` : The organization identifier.

Returns: A task that represents the asynchronous operation.

`M:Task_Weaver_Backend.Services.UserService.RegisterUserAsync(System.String,Task_Weaver_Backend.Models.DTos.RegisterUserDto)`
Performs register user async.

Parameter

- `uid` : The user identifier.
- `registrationData` : The registration data.

Returns: A task that represents the asynchronous operation.

`M:Task_Weaver_Backend.Services.UserService.UpdateUserAsync(System.String,Task_Weaver_Backend.Models.UserDto)`
Performs update user async.

Parameter

- `uid` : The user identifier.
- `updateData` : The update data.

Returns: A task that represents the asynchronous operation.

Task_Weaver_Backend.Services.WorkProfileService

Handles working schedules and chunk settings for a user work profile.

`M:Task_Weaver_Backend.Services.WorkProfileService.#ctor(Task_Weaver_Backend.Services.IScheduler,Task_Weaver_Backend.Models.IWorkProfileRepository,Ta`
Initializes a new instance of the

Parameter

- `scheduler` : Scheduler used to re-plan tasks after profile changes.
- `workProfileRepository` : Repository for persisting work profile data.
- `workProfileValidator` : Validator for schedules and chunk settings.
- `organizationRepository` : Repository used to resolve organization membership.
- `userRepository` : Repository used to resolve user data.

`M:Task_Weaver_Backend.Services.WorkProfileService.CreateWorkingScheduleAsync(System.String,Task_Weaver_Backend.Models.WorkingScheduleDto)`
Creates a new working schedule and triggers task re-scheduling.

Parameter

- `uid` : The user identifier.
- `schedule` : Working schedule payload including year, week range, and day slots.

Returns: A task that represents the asynchronous operation.

`M:Task_Weaver_Backend.Services.WorkProfileService.DeleteWorkingScheduleAsync(System.String,System.String)`
Deletes a working schedule and triggers task re-scheduling.

Parameter

- `uid` : The user identifier.
- `scheduleId` : The schedule identifier.

Returns: A task that represents the asynchronous operation.

`M:Task_Weaver_Backend.Services.WorkProfileService.GetChunkSettingsAsync(System.String)`
Loads global chunk settings for the specified user profile. If no settings are stored yet, default values are returned.

Parameter

- `uid` : The user identifier.

Returns: Resolved global chunk settings for task planning.

`M:Task_Weaver_Backend.Services.WorkProfileService.GetWorkingSchedulesAsync(System.String)`
Loads all working schedules configured by the user.

Parameter

- `uid` : The user identifier.

Returns: List of working schedules for the user.

`M:Task_Weaver_Backend.Services.WorkProfileService.PatchChunkSettingsAsync(System.String,Task_Weaver_Backend.Models.GlobalChunkSettingsDto)`
Updates global chunk settings and triggers task re-scheduling.

Parameter

- `uid` : The user identifier.
- `settings` : Chunk planning constraints such as min/max chunk duration and defaults.

Returns: A task that represents the asynchronous operation.

`M:Task_Weaver_Backend.Services.WorkProfileService.PatchWorkingScheduleAsync(System.String,System.String,Task_Weaver_Backend.Models.WorkingScheduleDto)`
Updates an existing working schedule and triggers task re-scheduling.

Parameter

- `uid` : The user identifier.
- `scheduleId` : The schedule identifier.
- `schedule` : Updated schedule payload that replaces the existing schedule values.

Returns: A task that represents the asynchronous operation.

Task_Weaver_Backend.Validators.BlockerValidator

Represents the blocker validator.

`M:Task_Weaver_Backend.Validators.BlockerValidator.ValidateBlocker(Task_Weaver_Backend.Models.BlockerDto)`
Performs validate blocker.

Parameter

- `blocker` : The blocker.

Task_Weaver_Backend.Validators.IBlockerValidator

interface IBlockerValidator.

`M:Task_Weaver_Backend.Validators.IBlockerValidator.ValidateBlocker(Task_Weaver_Backend.Models.BlockerDto)`
Performs validate blocker.

Parameter

- `blocker` : The blocker.

Task_Weaver_Backend.Validators.IOrganizationValidator

interface IOrganizationValidator.


```
M:Task_Weaver_Backend.Validators.IOrganizationValidator.ValidateUserIsAdminOfOrganization(System.String,System.String,System.Collections.Generic.List<System.String>)
Performs validate user is admin of organization.
```

Parameter

- `uid` : The user identifier.
- `orgId` : The organization identifier.
- `admins` : The admins.

```
M:Task_Weaver_Backend.Validators.IOrganizationValidator.ValidateUserIsNotLastAdminOfOrganization(System.String,System.String,System.Collections.Generic.List<System.String>)
Performs validate user is not last admin of organization.
```

Parameter

- `orgId` : The organization identifier.
- `targetUid` : The user identifier.
- `admins` : The admins.

Task_Weaver_Backend.Validators.IShearedValidator

interface IShearedValidator.

```
M:Task_Weaver_Backend.Validators.IShearedValidator.ValidateUserIsMemberOfOrganization(System.String,System.String,System.Collections.Generic.List<System.String>)
Performs validate user is member of organization.
```

Parameter

- `uid` : The user identifier.
- `orgId` : The organization identifier.
- `members` : The members.

Task_Weaver_Backend.Validators.ITaskValidator

interface ITaskValidator.

```
M:Task_Weaver_Backend.Validators.ITaskValidator.ValidateCreateTaskDto(Task_Weaver_Backend.Models.DTos.CreateTaskDto,System.String,System.Collections.Generic.List<System.String>)
Performs validate create task dto.
```

Parameter

- `taskData` : The task data.
- `uid` : The user identifier.
- `userOrganizations` : The user organizations.

```
M:Task_Weaver_Backend.Validators.ITaskValidator.ValidateTaskDto(Task_Weaver_Backend.Models.DTos.TaskDto,System.String,System.Collections.Generic.List<System.String>)
Performs validate task dto.
```

Parameter

- `taskData` : The task data.
- `uid` : The user identifier.
- `userOrganizations` : The user organizations.

Task_Weaver_Backend.Validators.IUserValidator

interface IUserValidator.

```
M:Task_Weaver_Backend.Validators.IUserValidator.ValidateUserDto(System.String,Task_Weaver_Backend.Models.UserDto)
Performs validate user dto.
```

Parameter

- `uid` : The user identifier.
- `userDto` : The user dto.

Task_Weaver_Backend.Validators.IWorkProfileValidator

interface IWorkProfileValidator.

```
M:Task_Weaver_Backend.Validators.IWorkProfileValidator.ValidateChunkSettings(System.String,Task_Weaver_Backend.Models.GlobalChunkSettingsDto)
Performs validate chunk settings.
```

Parameter

- `uid` : The user identifier.
- `settings` : The settings.

```
M:Task_Weaver_Backend.Validators.IWorkProfileValidator.ValidateWorkingSchedule(Task_Weaver_Backend.Models.WorkingScheduleDto,System.Collections.Generic.List<System.String>)
Performs validate working schedule.
```

Parameter

- `schedule` : The schedule.
- `oldWorkingSchedules` : The old working schedules.

Task_Weaver_Backend.Validators.OrganizationValidator

Represents the organization validator.

```
M:Task_Weaver_Backend.Validators.OrganizationValidator.ValidateUserIsAdminOfOrganization(System.String,System.String,System.Collections.Generic.List<System.String>)
Performs validate user is admin of organization.
```

Parameter

- `uid` : The user identifier.
- `orgId` : The organization identifier.
- `admins` : The admins.

```
M:Task_Weaver_Backend.Validators.OrganizationValidator.ValidateUserIsNotLastAdminOfOrganization(System.String,System.String,System.Collections.Generic.List<System.String>)
Performs validate user is not last admin of organization.
```

Parameter

- `orgId` : The organization identifier.
- `targetUid` : The user identifier.
- `admins` : The admins.

Task_Weaver_Backend.Validators.ShearedValidator

Represents the sheared validator.

```
M:Task_Weaver_Backend.Validators.ShearedValidator.ValidateUserIsMemberOfOrganization(System.String,System.String,System.Collections.Generic.List<System.String>)
Performs validate user is member of organization.
```

Parameter

- `uid` : The user identifier.
- `orgId` : The organization identifier.
- `members` : The members.

Task_Weaver_Backend.Validators.TaskValidator

Represents the task validator.

```
M:Task_Weaver_Backend.Validators.TaskValidator.ValidateCreateTaskDto(Task_Weaver_Backend.Models.DTOS.CreateTaskDto,System.String,System.Collections.Generic.List<System.String>)
Performs validate create task dto.
```

Parameter

- `taskData` : The task data.
- `uid` : The user identifier.
- `userOrganizations` : The user organizations.

```
M:Task_Weaver_Backend.Validators.TaskValidator.ValidateTaskDto(Task_Weaver_Backend.Models.DTOS.TaskDto,System.String,System.Collections.Generic.List<System.String>)
Performs validate task dto.
```

Parameter

- `taskData` : The task data.
- `uid` : The user identifier.
- `userOrganizations` : The user organizations.

Task_Weaver_Backend.Validators.UserValidator

Represents the user validator.

```
M:Task_Weaver_Backend.Validators.UserValidator.ValidateUserDto(System.String,Task_Weaver_Backend.Models.UserDto)
Performs validate user dto.
```

Parameter

- `uid` : The user identifier.
- `userDto` : The user dto.

Task_Weaver_Backend.Validators.WorkProfileValidator

Represents the work profile validator.

```
M:Task_Weaver_Backend.Validators.WorkProfileValidator.ValidateChunkSettings(System.String,Task_Weaver_Backend.Models.GlobalChunkSettingsDto)
Performs validate chunk settings.
```

Parameter

- `uid` : The user identifier.
- `settings` : The settings.

```
M:Task_Weaver_Backend.Validators.WorkProfileValidator.ValidateWorkingSchedule(Task_Weaver_Backend.Models.WorkingScheduleDto,System.Collections.Generic.List<System.String>)
Performs validate working schedule.
```

Performs validate working schedule.

Parameter

- `schedule` : The schedule.
- `oldWorkingSchedules` : The old working schedules.